TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN

[Họ và tên thành viên 1] **– 52400062**
[Họ và tên thành viên 2] **– 52400200**
[Họ và tên thành viên 3] **– 52400063**
[Họ và tên thành viên 4] **– 52400248**

BÁO CÁO CUỐI KỲ

NHẬP MÔN XỬ LÝ ẢNH SỐ (505060)

PHÁT HIỆN, THEO DÕI VÀ NHẬN DẠNG NỘI DUNG BIỂN BÁO GIAO THÔNG TRONG VIDEO

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

TỔNG LIÊN ĐOÀN LAO ĐỘNG VIỆT NAM
TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN

[Họ và tên thành viên 1] **– 52400062**
[Họ và tên thành viên 2] **– 52400200**
[Họ và tên thành viên 3] **– 52400063**
[Họ và tên thành viên 4] **– 52400248**

BÁO CÁO CUỐI KỲ

NHẬP MÔN XỬ LÝ ẢNH SỐ (505060)

PHÁT HIỆN, THEO DÕI VÀ NHẬN DẠNG NỘI DUNG BIỂN BÁO GIAO THÔNG TRONG VIDEO

Người hướng dẫn: [ThS/TS Họ tên giảng viên]

THÀNH PHỐ HỒ CHÍ MINH, NĂM 2026

LỜI CẢM ƠN

Nhóm chúng em xin gửi lời cảm ơn chân thành đến giảng viên hướng dẫn môn Nhập môn Xử lý ảnh số đã tận tình truyền đạt kiến thức nền tảng về xử lý ảnh – từ không gian màu, lọc ảnh, phép toán hình thái học đến phân tích contour – và đưa ra những góp ý quý báu trong quá trình thực hiện bài tiểu luận giữa kỳ, giúp nhóm hoàn thiện hệ thống trong báo cáo cuối kỳ này.

Chúng em cũng cảm ơn Khoa Công nghệ Thông tin, Trường Đại học Tôn Đức Thắng đã tạo môi trường học tập thuận lợi. Do thời gian và kiến thức còn hạn chế, báo cáo khó tránh khỏi thiếu sót; nhóm rất mong nhận được ý kiến đóng góp của thầy/cô để hoàn thiện hơn.

TP. Hồ Chí Minh, ngày [..] *tháng 11 năm 2026*

Nhóm sinh viên thực hiện

CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG

Chúng tôi xin cam đoan đây là sản phẩm của nhóm chúng tôi dưới sự hướng dẫn của giảng viên môn học. Các nội dung, mã nguồn và kết quả trong đề tài này là trung thực và chưa được công bố dưới bất kỳ hình thức nào trước đây. Những số liệu, hình ảnh phục vụ cho việc phân tích, nhận xét, đánh giá được chính nhóm thực hiện từ hai video của đề bài. Ngoài ra, báo cáo có sử dụng một số tài liệu tham khảo, đều được trích dẫn và ghi rõ nguồn gốc.

Nếu phát hiện có bất kỳ sự gian lận nào, chúng tôi xin hoàn toàn chịu trách nhiệm về nội dung báo cáo của mình. Trường Đại học Tôn Đức Thắng không liên quan đến những vi phạm tác quyền, bản quyền do chúng tôi gây ra trong quá trình thực hiện (nếu có).

TP. Hồ Chí Minh, ngày [..] *tháng 11 năm 2026*
Tác giả (ký tên và ghi rõ họ tên)
[Họ và tên thành viên 1]
[Họ và tên thành viên 2]
[Họ và tên thành viên 3]
[Họ và tên thành viên 4]

PHẦN XÁC NHẬN VÀ ĐÁNH GIÁ CỦA GIẢNG VIÊN

Phần xác nhận của GV hướng dẫn

……
……
……
……
……
……
……
……
……
……
……
……

TP. Hồ Chí Minh, ngày … tháng … năm 2026
(ký và ghi họ tên)

Phần đánh giá của GV chấm bài

……
……
……
……
……
……
……
……
……
……
……
……

TP. Hồ Chí Minh, ngày … tháng … năm 2026
(ký và ghi họ tên)

TÓM TẮT

Báo cáo trình bày hệ thống tự động **phát hiện, theo dõi và nhận dạng nội dung biển báo giao thông** trong hai video quay trên đường phố, chỉ sử dụng OpenCV, NumPy và math, không dùng mô hình huấn luyện sẵn. Biển báo được phát hiện bằng phân ngưỡng màu HSV (đỏ, xanh, vàng), phép toán hình thái học và kiểm tra hình học contour cho ba dạng: tròn (biển cấm, hiệu lệnh), tam giác (cảnh báo) và chữ nhật (chỉ dẫn); ảnh được phóng to đa tỉ lệ để bắt biển ở xa. Các ứng viên được ghép thành track bằng dự đoán chuyển động tuyến tính, sau đó lọc bằng thống kê trên toàn track (vị trí, độ tròn, màu, xu hướng to dần) để loại nhiễu, nối các mảnh bị đứt, khử trùng, nội suy và làm mượt để box ổn định và duy nhất từ xa đến gần. Nội dung biển được suy ra từ tỉ lệ màu theo vùng, phân tích thành phần liên thông và so khớp mẫu với các ký hiệu vẽ bằng code, rồi bỏ phiếu trên các quan sát gần nhất.

Trên hai video của đề bài, hệ thống xuất ra 31 biển báo (26 ở video 1 và 5 ở video 2) với nội dung nhận dạng được hiển thị ngay trên mỗi biển; video kết quả giữ nguyên kích thước, tốc độ khung hình, số frame và hệ màu của video gốc. Chương 1 trình bày chi tiết từng bước xử lý kèm mã nguồn và hình minh hoạ; Chương 2 trình bày kết quả, các frame nhận dạng chưa chính xác và đường dẫn video kết quả.

MỤC LỤC

LỜI CẢM ƠN ..... **i**
CÔNG TRÌNH ĐƯỢC HOÀN THÀNH TẠI TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG ..... **ii**
PHẦN XÁC NHẬN VÀ ĐÁNH GIÁ CỦA GIẢNG VIÊN ..... **iii**
TÓM TẮT ..... **iv**
DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT ..... **vi**
DANH MỤC HÌNH ..... **vii**
DANH MỤC BẢNG ..... **vii**
CHƯƠNG 1 – PHƯƠNG PHÁP GIẢI QUYẾT BÀI TOÁN **1**
1.1 Phát biểu bài toán và hướng tiếp cận 1
1.2 Bước 1 – Đọc video, cấu hình và vùng quan tâm (ROI) 2
1.3 Bước 2 – Tiền xử lý chống nhiễu và cân bằng sáng 3
1.4 Bước 3 – Phân ngưỡng màu HSV và phép toán hình thái học 4
1.5 Bước 4 – Tìm contour và kiểm tra hình dạng 5
1.5.1 Ứng viên hình tròn (biển cấm, biển hiệu lệnh) 5
1.5.2 Ứng viên tam giác (biển cảnh báo) 7
1.5.3 Ứng viên chữ nhật (biển chỉ dẫn nền xanh) 8
1.6 Bước 5 – Đa tỉ lệ cho biển ở xa, gộp ứng viên và cắt ảnh chuẩn hoá 9
1.7 Bước 6 – Theo dõi đối tượng (tracking) trực tuyến 11
1.8 Bước 7 – Hậu xử lý track: phân loại, nối mảnh, khử trùng 12
1.9 Bước 8 – Nhận dạng nội dung biển báo 14
1.9.1 Tách mask màu trên ảnh chuẩn hoá 14
1.9.2 Biển tròn: phân nhóm theo màu và so khớp mẫu 15
1.9.3 Biển tam giác và biển chữ nhật 18
1.9.4 Bỏ phiếu trên track 19
1.10 Bước 9 – Nội suy và làm mượt box 20
1.11 Bước 10 – Ghi video kết quả và kiểm tra 21
CHƯƠNG 2 – KẾT QUẢ THỰC HIỆN **24**
2.1 Tổng hợp kết quả 24
2.2 Các frame nhận dạng chưa chính xác – Video 1 25
2.3 Các frame nhận dạng chưa chính xác – Video 2 27
2.4 Đường dẫn video kết quả 28
2.5 Nhận xét và hướng phát triển 28
TÀI LIỆU THAM KHẢO **30**

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Chữ viết tắt	Ý nghĩa
HSV	Hue – Saturation – Value (không gian màu sắc độ – bão hoà – độ sáng)
ROI	Region of Interest – vùng quan tâm
CLAHE	Contrast Limited Adaptive Histogram Equalization
FPS	Frames per second – số khung hình mỗi giây
IoU	Intersection over Union – tỉ lệ giao trên hợp
MSSV	Mã số sinh viên
HCN	Hình chữ nhật

DANH MỤC HÌNH

Hình 1.1: Sơ đồ tổng quát của hệ thống (hai lượt).....2
Hình 1.2: ROI (vàng) và hai vùng phóng to ×2 (tím), ×3 (cam) trên một frame của video 1.....3
Hình 1.3: Ảnh gốc, sau lọc bilateral, kênh V và kênh V sau CLAHE (biển nằm trong bóng cây được tăng tương phản).....4
Hình 1.4: Mask đỏ, mask đỏ sau Open + Close, mask xanh (video 1) và mask vàng (video 2).....5
Hình 1.5: Ứng viên phát hiện trên một frame của video 1 (xanh lá: tròn, cam: chữ nhật). Còn có ứng viên sai – sẽ bị loại ở bước theo dõi.....9
Hình 1.6: Ứng viên tam giác (vàng) và tròn (xanh lá) trên video 2.....9
Hình 1.7: Biển ở xa được bắt nhờ ảnh phóng to (đỏ: ứng viên bán kính < 4 px). Các ô trên biển quảng cáo là ứng viên sai, sẽ bị loại khi lọc track.....11
Hình 1.8: Patch chuẩn hoá và 5 mask màu của các loại biển: hiệu lệnh, cấm rẽ, cấm dừng đỗ, cảnh báo, chỉ dẫn làn.....15
Hình 1.9: Bộ mẫu ký hiệu được vẽ bằng code: 7 mũi tên biển hiệu lệnh và 3 ký hiệu biển cấm rẽ.....16
Hình 1.10: Kích thước và vị trí box của một track: điểm đỏ – detection thô, đường xanh – sau nội suy và làm mượt.....20
Hình 1.11: Một frame kết quả của video 1: box, nội dung biển và MSSV.....22
Hình 1.12: Một frame kết quả của video 2.....23
Hình 2.1: Video 1, frame 150 (t = 5.0 s): hai biển cấm rẽ trái chưa được phát hiện (khoanh đỏ).....26
Hình 2.2: Video 1, frame 2280 (t = 76.1 s): biển “CHÚ Ý QUAN SÁT” nền đỏ không được phát hiện.....26
Hình 2.3: Video 2, frame 30 (t = 1.0 s): các biển trong đoạn chuyển cảnh không được phát hiện.....27
Hình 2.4: Video 2, frame 470 (t = 15.7 s): hai biển cảnh báo ở xa chưa được phát hiện.....28

DANH MỤC BẢNG

Bảng 2.1: Các biển báo được phát hiện và nhận dạng.....24

CHƯƠNG 1 – PHƯƠNG PHÁP GIẢI QUYẾT BÀI TOÁN

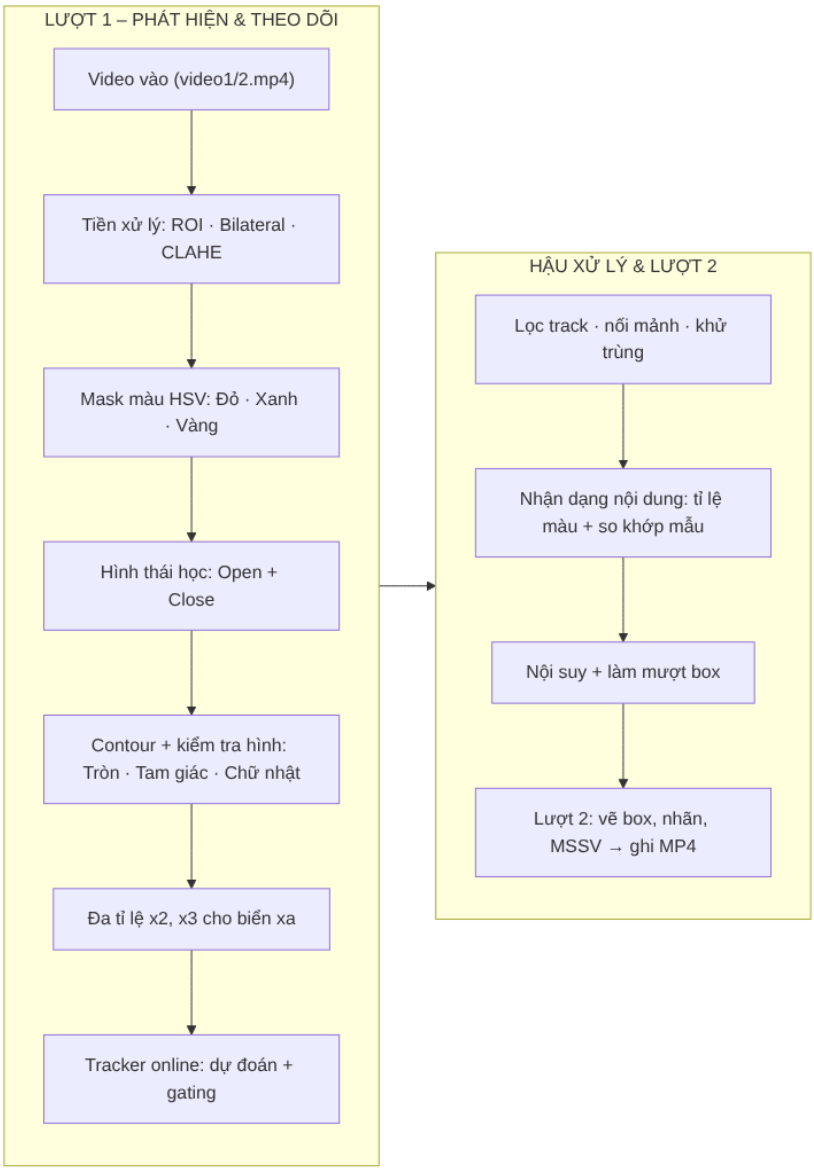
1.1 Phát biểu bài toán và hướng tiếp cận

Đầu vào là hai video quay từ xe máy trên đường phố TP.HCM (`video1.mp4` – 3087 frame, `video2.mp4` – 2278 frame, cùng độ phân giải 1920×1080, 29,97 fps). Yêu cầu: vẽ hình chữ nhật bao quanh **mọi biển báo giao thông ở giữa hoặc bên phải đường** trong tất cả các frame, **suy ra nội dung biển** và ghi nội dung ngay phía trên/dưới biển, rồi lưu thành video MP4 mới có cùng hệ màu, kích thước frame và tốc độ khung hình với video gốc; MSSV phải xuất hiện trên mọi frame.

Ràng buộc của đề: chỉ dùng **OpenCV, NumPy và math**; không dùng mô hình huấn luyện sẵn; không hardcode theo frame/video; dùng **cùng một mã nguồn và cùng một bộ tham số** cho cả hai video. Vì vậy nhóm xây dựng một hệ thống thuần xử lý ảnh cổ điển, gồm 3 khối chính:

- **Phát hiện (detection):** tách màu đặc trưng của biển báo (đỏ, xanh, vàng) trong không gian HSV, làm sạch bằng phép toán hình thái học, tìm contour và kiểm tra hình dạng hình học (tròn, tam giác, chữ nhật).
- **Theo dõi (tracking):** ghép các ứng viên giữa các frame thành "track", sau đó lọc track bằng các đặc trưng thống kê trên toàn bộ thời gian xuất hiện để loại vật thể không phải biển báo; nội suy và làm mượt box để box ổn định và duy nhất từ xa đến gần.
- **Nhận dạng nội dung (recognition):** chuẩn hoá ảnh biển về 64×64, tính tỉ lệ màu theo vùng, phân tích thành phần liên thông và **so khớp mẫu (template matching)** với các ký hiệu được vẽ bằng code; bỏ phiếu trên các quan sát gần nhất của track.

Toàn bộ chương trình chạy theo cơ chế **hai lượt (two-pass)**: lượt 1 đọc video để phát hiện và theo dõi; sau khi có đầy đủ track thì hậu xử lý (lọc, nối, nhận dạng, làm mượt); lượt 2 đọc lại video, vẽ kết quả và ghi file. Cách làm này cho phép dùng thông tin "tương lai" của track (ví dụ biển to dần khi xe tiến lại gần) để quyết định một vật thể có phải biển báo hay không – điều mà xử lý từng frame độc lập không làm được. Hình 1.1 tóm tắt toàn bộ quy trình.



Hình 1.1: Sơ đồ tổng quát của hệ thống (hai lượt)

1.2 Bước 1 – Đọc video, cấu hình và vùng quan tâm (ROI)

Chương trình mở video bằng `cv2.VideoCapture`, lấy FPS và số frame từ chính file (không gán cứng 30 fps như mã mẫu) để video kết quả có **cùng FPS** với video gốc. Mọi tham số được khai báo một lần ở đầu file và dùng chung cho cả hai video. Các ngưỡng màu HSV được chọn rộng để chịu được thay đổi ánh sáng; riêng màu xanh dùng S và V cao để tách biển xanh khỏi các vùng xanh tối (cây, kính) có cùng Hue.

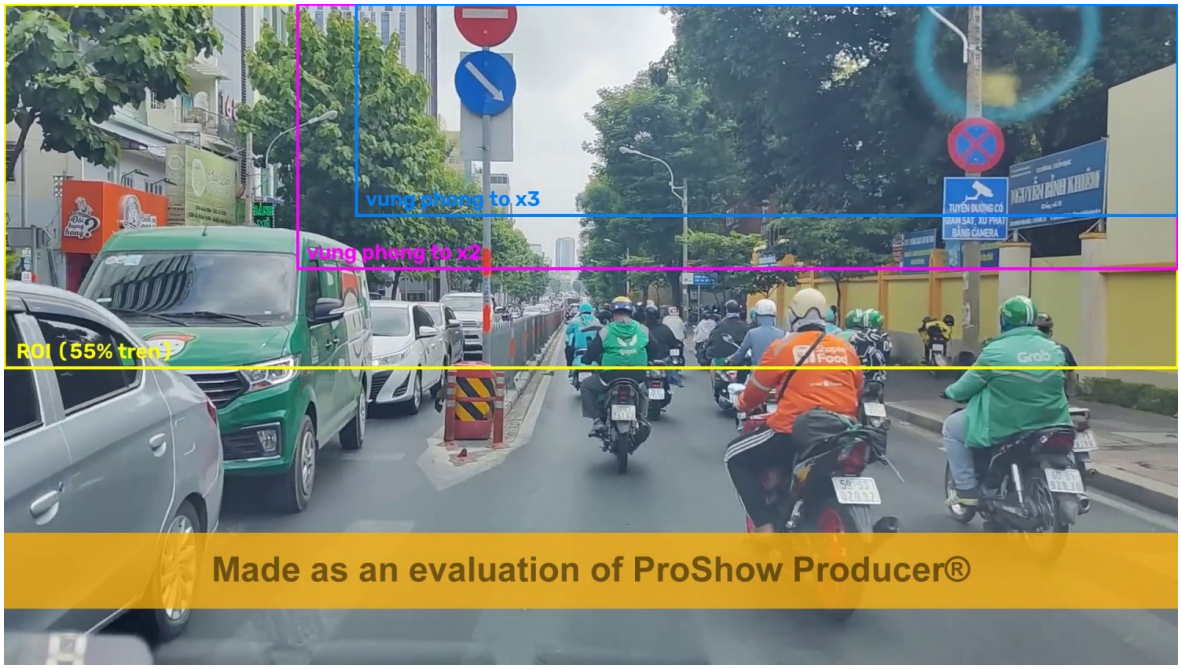
Mã nguồn: phần cấu hình

```
# ---- Mau (HSV cua OpenCV: H 0..180, S 0..255, V 0..255) ----
# Mau do nam hai dau vong Hue. Dai Hue rong de chiu duoc thay doi anh sang.
RED_H_LOW, RED_H_HIGH = 5, 150
RED_S_MIN, RED_V_MIN = 70, 50
# Mau xanh bien bao: S cao va V cao de tach khoi vung xanh toi co cung Hue.
BLUE_H_MIN, BLUE_H_MAX = 98, 130
BLUE_S_MIN, BLUE_V_MIN = 135, 115
# Mau vang bien canh bao.
```

```
YEL_H_MIN, YEL_H_MAX = 14, 34
YEL_S_MIN, YEL_V_MIN = 110, 120

# ---- Bien TAM GIAC canh bao (long vang, vien do, dinh huong len) ----
TRI_AR = (0.80, 1.60)       # w / h cua long vang (tam giac deu ~ 1.15)
TRI_FILL = (0.38, 0.66)     # dien tich bao loi / (w*h): tam giac ly tuong = 0.5
TRI_APEX_DX = 0.22          # dinh tren lech tam toi da (ti le w)
TRI_EXPAND = 1.30           # no box long vang ra de bao ca vien do
# ---- Bien CHU NHAT chi dan (nen xanh) ----
RECT_AR = (0.70, 2.60)      # w / h
RECT_FILL = 0.82            # dien tich bao loi / (w*h): chu nhat ~ 1, tron ~ 0.785
RECT_CORNER = 0.15          # o vuong goc (ti le canh) phai duoc bao loi phu kin (>= 3/4
goc)
RECT_BLUE_MIN = 0.35        # ti le pixel xanh toi thieu trong HCN
RECT_MIN_AREA = 120         # px (he goc): bien chu nhat qua nho khong doc duoc noi dung
PATCH = 64                  # kich thuoc anh chuan hoa cua 1 ung vien (de nhan dang)
PATCH_KEEP = 6              # moi track giu PATCH_KEEP anh lon nhat de bo phieu noi dung
```

Biển báo cần nhận dạng nằm ở giữa hoặc bên phải đường và luôn ở nửa trên khung hình, nên việc tìm kiếm chỉ thực hiện trên **55% phía trên** của frame (ROI). Để bắt các biển rất xa (chỉ vài pixel), hai vùng con ở dải trên – phải được phóng to ×2 và ×3 rồi tìm lại (xem mục 1.6). Hình 1.2 minh hoạ các vùng này.



Hình 1.2: ROI (vàng) và hai vùng phóng to ×2 (tím), ×3 (cam) trên một frame của video 1

1.3 Bước 2 – Tiền xử lý chống nhiễu và cân bằng sáng

Video bị nén mạnh và có nhiều nhiễu khối; biển ở xa thì nhoè. Nhóm dùng **lọc song phương (bilateral filter)** – làm mịn vùng đồng màu nhưng giữ biên sắc nét (viền đỏ mảnh của biển xa không bị nhoè mất). Với ảnh đã phóng to ×3, bilateral quá chậm nên thay bằng Gaussian 3×3. Sau đó ảnh được chuyển sang HSV và áp dụng **CLAHE** (cân bằng histogram thích nghi có giới hạn tương phản) lên kênh V. Kênh V sau CLAHE chỉ

dùng để **tạo mask màu** (ổn định khi biển bị ngược sáng, nằm trong bóng cây); còn các đặc trưng đo độ sáng vẫn dùng kênh V gốc để không làm sai lệch giá trị thật.

Mã nguồn: _preprocess() — dòng 245–263 trong file .py

```
def _preprocess(img, scale=1.0):
    """
    Tra ve (hsv_goc, hsv_mask).
      hsv_goc  : dung de tinh dac trung (giu nguyen do sang that).
      hsv_mask : kenh V da CLAHE, chi dung de tao mask mau (on dinh voi anh sang yeu/chen).
    """
    if scale >= 3.0:          # anh phong to lon: bilateral qua cham, Gaussian nhe la du
        img = cv2.GaussianBlur(img, (3, 3), 0)
    elif USE_BILATERAL:
        img = cv2.bilateralFilter(img, BILATERAL_D, BILATERAL_SC, BILATERAL_SS)
    else:
        img = cv2.GaussianBlur(img, (5, 5), 0)
    hsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
    if USE_CLAHE:
        hsv_mask = hsv.copy()
        hsv_mask[:, :, 2] = _CLAHE.apply(hsv[:, :, 2])
    else:
        hsv_mask = hsv
    return hsv, hsv_mask
```



Hình 1.3: Ảnh gốc, sau lọc bilateral, kênh V và kênh V sau CLAHE (biển nằm trong bóng cây được tăng tương phản)

1.4 Bước 3 – Phân ngưỡng màu HSV và phép toán hình thái học

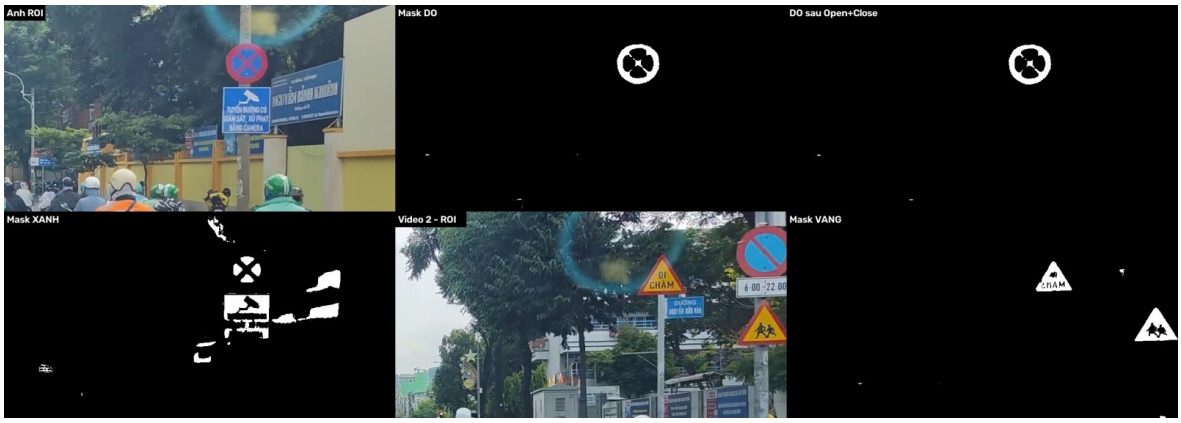
Biển báo Việt Nam dùng ba màu chủ đạo: **đỏ** (viền biển cấm, nền biển cấm đi ngược chiều, viền biển cảnh báo), **xanh lam** (biển hiệu lệnh, biển chỉ dẫn, nền biển cấm đỗ) và **vàng** (nền biển cảnh báo). Màu đỏ nằm ở hai đầu vòng Hue nên được ghép từ hai khoảng `[0, 5]` và `[150, 180]`. Mask vàng dùng H ∈ [14, 34] với S, V đủ cao để không lẫn với tường vàng nhạt.

Mã nguồn: build_color_masks() — dòng 176–182 trong file .py

```
def build_color_masks(hsv):
    """Tra ve 2 mask nhi phan (0/255): mau do va mau xanh bien bao."""
    red = cv2.inRange(hsv, (0, RED_S_MIN, RED_V_MIN), (RED_H_LOW, 255, 255))
    red |= cv2.inRange(hsv, (RED_H_HIGH, RED_S_MIN, RED_V_MIN), (180, 255, 255))
    blue = cv2.inRange(hsv, (BLUE_H_MIN, BLUE_S_MIN, BLUE_V_MIN),
                       (BLUE_H_MAX, 255, 255))
    return red, blue
```

Mask nhị phân sau phân ngưỡng còn lỗ thủng (do vạch trắng cắt ngang viền, do chữ trên biển) và các chấm nhiễu nhỏ. Nhóm dùng **MORPH_OPEN** với nhân rất nhỏ (2×2)

để xoá chấm nhiễu mà không xoá viền mảnh của biển xa, sau đó **MORPH_CLOSE** với nhân elip 5×5 để nối liền viền. Riêng mask đỏ được đóng thêm một lần với nhân 11×11 để nối các mảnh viền bị vạch trắng (biển cấm đi ngược chiều) cắt rời. Mỗi ảnh được xét với bốn mask: đỏ, xanh, đỏ ∪ xanh (biển cấm đỗ có cả hai màu) và đỏ với nhân lớn.



Hình 1.4: Mask đỏ, mask đỏ sau Open + Close, mask xanh (video 1) và mask vàng (video 2)

1.5 Bước 4 – Tìm contour và kiểm tra hình dạng

1.5.1 Ứng viên hình tròn (biển cấm, biển hiệu lệnh)

Với mỗi mask, `cv2.findContours` (chế độ `RETR_EXTERNAL`) trả về các đường bao ngoài. Mỗi contour được lấy bao lồi (convex hull) – bao lồi bù lại các khe hở do vạch trắng/chữ trên biển – rồi kiểm tra lần lượt:

- **Diện tích** trong khoảng [30, 90000] px (tính theo hệ toạ độ gốc) và cạnh box ≥ 5 px.
- **Tỉ lệ khung** w/h ∈ [0,60; 1,60] – cho phép biển nhìn xiên thành elip.
- **Độ lấp elip** `efill = S_baolôi / (π/4·w·h)`: hình tròn/elip ≈ 1,0; hình vuông ≈ 1,27; tam giác ≈ 0,64. Chỉ giữ efill ∈ [0,82; 1,12].
- **Độ đặc (solidity)** = diện tích contour / diện tích bao lồi ≥ 0,70. Với biển nhỏ (bán kính < 10 px) ảnh nhoè nên các ngưỡng được nới (efill ∈ [0,75; 1,18], solidity ≥ 0,62).

Ứng viên qua được bộ lọc hình dạng sẽ được tính **đặc trưng nội dung** theo toạ độ elip chuẩn hoá `d = √(((x−cx)/a)² + ((y−cy)/b)²)` (d = 1 là mép biển): tỉ lệ đỏ/xanh trên vành `d ∈ [0,70; 0,95]`, tỉ lệ pixel khác màu viền trong lòng biển `d ≤ 0,55`, độ sáng trung bình lòng biển, tỉ lệ pixel tối/xanh/đỏ ở vành ngoài `d ∈ [1,2; 1,6]`… Các đặc trưng này không quyết định ngay ở từng frame mà được lấy **trung vị trên cả track** ở bước hậu xử lý (mục 1.8). Nếu đĩa xanh nằm trong một vành đỏ mảnh (biển cấm đỗ xe), box được nới ra 1,22 lần để bao trọn cả viền đỏ.

Mã nguồn: _find_candidates() — dòng 266–341 trong file .py

```
    cands = []
    # 4 mask sau phep dong hinh thai: do, xanh, do|xanh, va do voi kernel lon
    # (kernel lon noi cac manh bi dut do vach trang cat ngang vien).
```

```
    for mask, kernel in ((red, k_small), (blue, k_small),
                         (cv2.bitwise_or(red, blue), k_small), (red, k_big)):
        m = mask
        if k_open is not None:
            m = cv2.morphologyEx(m, cv2.MORPH_OPEN, k_open)
        m = cv2.morphologyEx(m, cv2.MORPH_CLOSE, kernel)
        contours, _ = cv2.findContours(m, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
        for cnt in contours:
            area = cv2.contourArea(cnt)
            if not (area_min <= area <= area_max):
                continue
            hull = cv2.convexHull(cnt)
            hull_area = cv2.contourArea(hull)
            if hull_area <= 0:
                continue
            x, y, w, h = cv2.boundingRect(hull)
            ar = w / float(h)
            if not (CAND_AR_RANGE[0] <= ar <= CAND_AR_RANGE[1]):
                continue
            efill = hull_area / (math.pi / 4.0 * w * h)
            per = cv2.arcLength(hull, True)
            circ = 4.0 * math.pi * hull_area / (per * per + 1e-6)
            solid = area / hull_area

            r_orig = math.sqrt(w * h) / 2.0 / scale
            if max_r is not None and r_orig >= max_r:
                continue
            # [A] nguong phu thuoc kich thuoc
            if r_orig < SMALL_R:
                ef_rng, solid_min = CAND_ELLIPSE_FILL_SMALL, CAND_MIN_SOLID_SMALL
            else:
                ef_rng, solid_min = CAND_ELLIPSE_FILL, CAND_MIN_SOLID
            if not (ef_rng[0] <= efill <= ef_rng[1]):
                continue
            if solid < solid_min:
                continue

            cx, cy, a, b = x + w / 2.0, y + h / 2.0, w / 2.0, h / 2.0
            f = ellipse_features(hsv, red, blue, cx, cy, a, b)
            # Neu dia xanh nam trong vien do mong: no box ra de bao ca vien.
            if f["border_blue"] > f["border_red"] and f["ring_red"] >= 0.35:
                a, b = a * 1.22, b * 1.22
            bx, by = max(0, int(round(cx - a))), max(0, int(round(cy - b)))
            bw = min(int(round(2 * a)), iw - bx)
            bh = min(int(round(2 * b)), ih - by)
            if bw < min_box or bh < min_box:
                continue
            f.update(box=(bx, by, bw, bh), cx=cx, cy=cy, r=math.sqrt(a * b),
                     efill=efill, solid=solid, ar=ar, circ=circ, shape="circle")
            cands.append(f)

    cands += _find_triangles(hsv, hsv_mask, red, scale, max_r, k_small)
    cands += _find_rectangles(hsv, blue, scale, max_r, k_small)
    return cands
```

1.5.2 Ứng viên tam giác (biển cảnh báo)

Biển cảnh báo có lòng **vàng** hình tam giác đều, đỉnh hướng lên, viền đỏ. Trên mask vàng, mỗi contour được kiểm tra: tỉ lệ khung ∈ [0,80; 1,60]; tỉ lệ lấp đầy HCN bao ∈ [0,38; 0,66] (tam giác lý tưởng = 0,5); `cv2.approxPolyDP` với ε = 7% chu vi phải cho **đúng 3 đỉnh**, trong đó một đỉnh nằm ở trên, gần giữa (lệch ≤ 22% chiều rộng) và hai đỉnh nằm ở đáy. Box của lòng vàng được nới 1,3 lần quanh trọng tâm (2/3 chiều cao tính từ đỉnh) để bao cả viền đỏ. Đặc trưng `tri_red` là tỉ lệ pixel đỏ trên dải viền giữa tam giác lớn và lòng vàng.

Mã nguồn: _find_triangles() — dòng 344–402 trong file .py

```
def _find_triangles(hsv, hsv_mask, red, scale, max_r, kernel):
    """
    Bien canh bao: long VANG hinh tam giac deu, dinh huong len, vien DO bao ngoai.
    Tim contour tren mask vang -> xap xi da giac -> kiem tra 3 dinh + vi tri dinh tren.
    """
    ih, iw = red.shape
    yel = cv2.inRange(hsv_mask, (YEL_H_MIN, YEL_S_MIN, YEL_V_MIN), (YEL_H_MAX, 255, 255))
    yel = cv2.morphologyEx(yel, cv2.MORPH_CLOSE, kernel)
    contours, _ = cv2.findContours(yel, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    out = []
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if not (CAND_MIN_AREA * 0.6 * scale * scale <= area <= CAND_MAX_AREA * scale *
scale):
            continue
        hull = cv2.convexHull(cnt)
        hull_area = cv2.contourArea(hull)
        x, y, w, h = cv2.boundingRect(hull)
        if hull_area <= 0 or h == 0:
            continue
        ar, fill = w / float(h), hull_area / float(w * h)
        if not (TRI_AR[0] <= ar <= TRI_AR[1] and TRI_FILL[0] <= fill <= TRI_FILL[1]):
            continue
        per = cv2.arcLength(hull, True)
        approx = cv2.approxPolyDP(hull, 0.07 * per, True)
        if len(approx) != 3:
            continue
        pts = approx.reshape(-1, 2).astype(np.float64)
        top = pts[np.argmin(pts[:, 1])]
        if abs(top[0] - (x + w / 2.0)) > TRI_APEX_DX * w or top[1] > y + 0.2 * h:
            continue                                # phai co 1 dinh o tren, chinh giua
        if np.sum(pts[:, 1] > y + 0.7 * h) != 2:
            continue                                # va 2 dinh o day
        r_orig = math.sqrt(w * h) / 2.0 / scale
        if max_r is not None and r_orig >= max_r:
            continue
        # no box quanh trong tam (2/3 chieu cao tinh tu dinh) de bao ca vien do
        gx, gy = x + w / 2.0, y + 2.0 * h / 3.0
        w2, h2 = w * TRI_EXPAND, h * TRI_EXPAND
        bx, by = int(round(gx - w2 / 2)), int(round(gy - 2.0 * h2 / 3.0))
        bx, by = max(0, bx), max(0, by)
        bw, bh = min(int(round(w2)), iw - bx), min(int(round(h2)), ih - by)
        if bw < CAND_MIN_BOX * scale or bh < CAND_MIN_BOX * scale:
            continue
        # ... (luoc bot) ...
        rr = red[by:by + bh, bx:bx + bw] > 0
```

```
        v = hsv[by:by + bh, bx:bx + bw, 2]
        inn = inner > 0
        out.append(dict(
            shape="tri", box=(bx, by, bw, bh), cx=bx + bw / 2.0, cy=by + bh / 2.0,
            r=math.sqrt(bw * bh) / 2.0, efill=fill, solid=area / hull_area, ar=ar,
circ=0.0,
            tri_red=float((rr & ring).sum()) / max(1, int(ring.sum())),
            mean_v=float(v[inn].mean()) if inn.any() else 0.0))
    return out
```

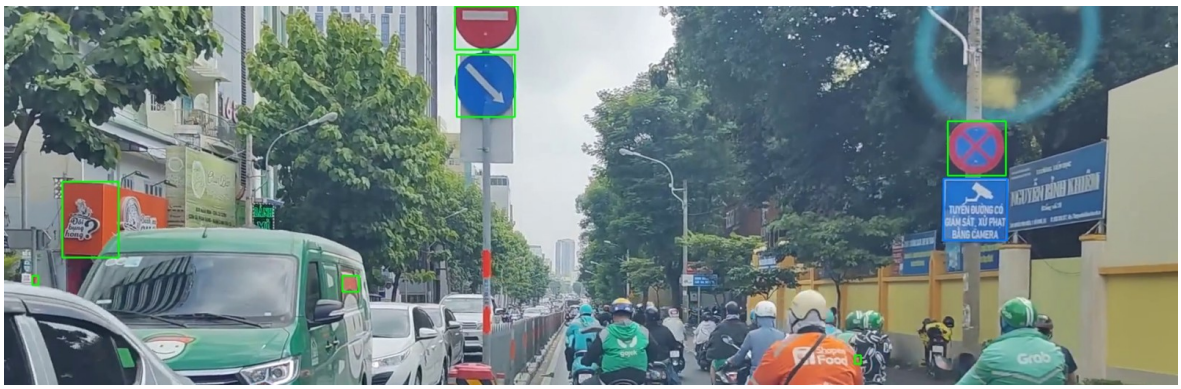
1.5.3 Ứng viên chữ nhật (biển chỉ dẫn nền xanh)

Biển chỉ dẫn (hướng đi theo làn, camera giám sát, đoạn đường hay xảy ra tai nạn, biển chỉ hướng trên giá long môn) là hình chữ nhật nền xanh. Contour trên mask xanh được lấy bao lồi; ứng viên phải có tỉ lệ khung $\in [0{,}70; 2{,}60]$, bao lồi lấp $\geq 82\%$ HCN bao, và **ít nhất 3/4 góc của HCN bao phải được lấp đầy** – hình tròn/elip luôn để trống cả bốn góc nên bị loại, còn biển chữ nhật bị che một góc (cột điện, cành cây) vẫn được giữ. Ngoài ra nhóm đo tỉ lệ pixel trắng ở dải ngay **bên trái** biển (`left_white`), dùng để nhận ra biển có ô hình minh hoạ màu trắng.

Mã nguồn: _find_rectangles() — dòng 405–456 trong file .py

```
def _find_rectangles(hsv, blue, scale, max_r, kernel):
    """
    Bien chi dan: hinh CHU NHAT nen XANH, ky hieu / chu mau trang ben trong.
    Contour ngoai cua mask xanh lap day gan tron HCN bao (fill >= RECT_FILL).
    """
    ih, iw = blue.shape
    m = cv2.morphologyEx(blue, cv2.MORPH_CLOSE, kernel)
    contours, _ = cv2.findContours(m, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE)
    out = []
    for cnt in contours:
        area = cv2.contourArea(cnt)
        if not (RECT_MIN_AREA * scale * scale <= area <= CAND_MAX_AREA * scale * scale):
            continue
        hull = cv2.convexHull(cnt)
        x, y, w, h = cv2.boundingRect(hull)
        ar, fill = w / float(h), cv2.contourArea(hull) / float(w * h)
        if not (RECT_AR[0] <= ar <= RECT_AR[1]) or fill < RECT_FILL:
            continue
        r_orig = math.sqrt(w * h) / 2.0 / scale
        if max_r is not None and r_orig >= max_r:
            continue
        # 4 goc cua HCN bao phai duoc lap day (hinh tron / elip de trong 4 goc)
        hm = np.zeros((h, w), np.uint8)
        cv2.fillConvexPoly(hm, (hull.reshape(-1, 2) - [x, y]).astype(np.int32), 1)
        cw, ch = max(1, int(w * RECT_CORNER)), max(1, int(h * RECT_CORNER))
        corners = (hm[:ch, :cw], hm[:ch, -cw:], hm[-ch:, :cw], hm[-ch:, -cw:])
        if sorted(float(c.mean()) for c in corners)[1] < 0.35:
            continue                                # >= 3/4 goc day (cho phep 1 goc bi che)
        blue_frac = float((blue[y:y + h, x:x + w] > 0).mean())
        if blue_frac < RECT_BLUE_MIN:
            continue
        mx, my = max(1, w // 10), max(1, h // 10)
        sub = hsv[y + my:y + h - my, x + mx:x + w - mx]
        if sub.size == 0:
```

```
            continue
        # o trang ngay BEN TRAI bien (hinh minh hoa cua bien canh bao dang chu nhat,
        # vd "Doan duong hay xay ra tai nan"): nguong trang tuong doi theo nen xanh
        box_hsv = hsv[y:y + h, x:x + w]
        bmask = blue[y:y + h, x:x + w] > 0
        s_blue = float(np.median(box_hsv[:, :, 1][bmask])) if bmask.any() else 150.0
        v_ref = float(np.percentile(box_hsv[:, :, 2], 97))
        lx = max(0, x - int(0.8 * h))
        strip = hsv[y + my:y + h - my, lx:x]
        left_white = float(((strip[:, :, 1] < max(80.0, 0.55 * s_blue))
                            & (strip[:, :, 2] >= max(110.0, 0.60 * v_ref))).mean()) \
            if strip.size else 0.0
        out.append(dict(
            shape="rect", box=(x, y, w, h), cx=x + w / 2.0, cy=y + h / 2.0,
            r=math.sqrt(w * h) / 2.0, efill=fill, solid=1.0, ar=ar, circ=0.0,
            rect_blue=blue_frac, left_white=left_white,
            mean_v=float(sub[:, :, 2].mean())))
    return out
```



Hình 1.5: Ứng viên phát hiện trên một frame của video 1 (xanh lá: tròn, cam: chữ nhật). Còn có ứng viên sai – sẽ bị loại ở bước theo dõi



Hình 1.6: Ứng viên tam giác (vàng) và tròn (xanh lá) trên video 2

1.6 Bước 5 – Đa tỉ lệ cho biển ở xa, gộp ứng viên và cắt ảnh chuẩn hoá

Biển ở xa chỉ rộng 6–12 px, viền đỏ mảnh 1–2 px nên dễ mất sau khi lọc. Để box xuất hiện **sớm và ổn định từ xa**, dải trên – phải của frame được phóng to ×2 (vùng y <

40%, x > 25%) và ×3 (y < 32%, x > 30%) bằng nội suy bicubic, rồi chạy lại đúng hàm tìm ứng viên với **cùng các ngưỡng**; chỉ giữ ứng viên nhỏ (bán kính gốc < 14 px với ×2, < 8 px với ×3) và quy toạ độ về frame gốc. Sau đó:

- **Loại "khối dính":** một ứng viên lớn chứa ≥ 2 ứng viên nhỏ tách rời bên trong (hai biển đứng sát nhau bị dính thành một khối) thì bị bỏ – trừ khi vành ngoài của nó liền mạch một màu (biển có hoạ tiết bên trong).
- **Gộp trùng:** sắp xếp theo diện tích giảm dần, bỏ ứng viên nằm ≥ 50% trong một ứng viên lớn hơn đã giữ (độ đo IoU-min = giao / diện tích box nhỏ).
- **Cắt patch:** vùng box của mỗi ứng viên được resize về **64×64** để dùng cho nhận dạng nội dung.

Mã nguồn: detect_candidates() — dòng 473–535 trong file .py

```
def detect_candidates(frame):
    """
    Tra ve danh sach ung vien bien tron/elip trong 1 frame.
    Moi ung vien: {'box': (x, y, w, h), 'cx', 'cy', 'r', 'efill', ... dac trung}
    """
    fh, fw = frame.shape[:2]
    y_lim = int(fh * ROI_Y_MAX)
    roi = frame[:y_lim]

    cands = _find_candidates(roi, 1.0)

    # [D] Multi-scale: phong to dai tren cua frame de bat bien rat nho (xa).
    if MS_ENABLE:
        my = int(fh * MS_Y_MAX)
        mx = int(fw * MS_X_MIN)
        crop = frame[:my, mx:]
        if crop.size > 0:
            up = cv2.resize(crop, None, fx=MS_SCALE, fy=MS_SCALE,
                            interpolation=cv2.INTER_CUBIC)
            for c in _find_candidates(up, MS_SCALE, max_r=MS_MAX_R):
                cands.append(_to_original(c, MS_SCALE, mx, 0))

    # [N] tang x3 cho bien rat xa (cung nguong voi ban goc)
    if FAR_ENABLE:
        my3, mx3 = int(fh * FAR_Y_MAX), int(fw * FAR_X_MIN)
        crop3 = frame[:my3, mx3:]
        if crop3.size > 0:
            up3 = cv2.resize(crop3, None, fx=FAR_SCALE, fy=FAR_SCALE,
                             interpolation=cv2.INTER_CUBIC)
            for c in _find_candidates(up3, FAR_SCALE, max_r=FAR_MAX_R):
                cands.append(_to_original(c, FAR_SCALE, mx3, 0))

    # ... (luoc bot) ...

    # Gop ung vien trung nhau: giu ung vien lon nhat.
    cands.sort(key=_area, reverse=True)
    kept = []
    for c in cands:
        if all(iou_min(c["box"], k["box"]) < 0.5 for k in kept):
            kept.append(c)
    # Cat anh chuan hoa PATCH x PATCH cua moi ung vien (dung cho buoc nhan dang noi dung).
    for c in kept:
```

```
        x, y, w, h = c["box"]
        roi_c = frame[max(0, y):min(fh, y + h), max(0, x):min(fw, x + w)]
        c["patch"] = (cv2.resize(roi_c, (PATCH, PATCH), interpolation=cv2.INTER_AREA
                                 if w > PATCH else cv2.INTER_CUBIC) if roi_c.size else
None)
    return kept
```



Hình 1.7: Biển ở xa được bắt nhờ ảnh phóng to (đỏ: ứng viên bán kính < 4 px). Các ô trên biển quảng cáo là ứng viên sai, sẽ bị loại khi lọc track

1.7 Bước 6 – Theo dõi đối tượng (tracking) trực tuyến

Mỗi track lưu danh sách quan sát `(frame, ứng viên)`. Tại frame mới, vị trí và kích thước của từng track được **dự đoán** bằng hồi quy tuyến tính (`np.polyfit` bậc 1) trên 10 quan sát gần nhất – biển trên xe đang chạy dịch chuyển và to dần gần như tuyến tính trong khoảng thời gian ngắn. Một ứng viên chỉ được ghép vào track nếu (1) **cùng dạng hình** (tròn/tam giác/chữ nhật), (2) khoảng cách tâm tới vị trí dự đoán ≤ 0,8 lần kích thước box (+0,05 cho mỗi frame bị mất), (3) tỉ lệ kích thước ≤ 1,6. Các cặp (track, ứng viên) hợp lệ được ghép tham lam theo chi phí tăng dần (khoảng cách + độ lệch kích thước), đảm bảo mỗi ứng viên thuộc tối đa một track – đây là cơ sở để kết quả **duy nhất (unique)**.

Track được giữ “sống” tối đa 20 frame không có detection (biển bị xe che khuất); track kết thúc sớm khi vị trí dự đoán đi ra khỏi mép trái/phải/trên của khung hình. Để tiết kiệm bộ nhớ, mỗi track chỉ giữ 6 patch của các quan sát lớn nhất (gần nhất) và có box đúng tỉ lệ.

Mã nguồn: Track.predict() — dòng 570–583 trong file .py

```
    def predict(self, frame_idx):
        """Du doan (cx, cy, s) o frame_idx bang hoi quy tuyen tinh tren FIT_LAST quan sat
cuoi."""
        pts = self.obs[-self.FIT_LAST:]
        f = np.array([p[0] for p in pts], dtype=np.float64)
        v = np.array([_box_to_cs(p[1]["box"]) for p in pts], dtype=np.float64)
        s_med = float(np.median(v[-5:, 2]))
        if len(pts) < 3 or f[-1] - f[0] < 2:
```

```
            return v[-1, 0], v[-1, 1], s_med
        out = []
        for k in range(3):
            slope, icpt = np.polyfit(f, v[:, k], 1)
            out.append(slope * frame_idx + icpt)
        s_pred = min(max(out[2], 0.7 * s_med), 1.5 * s_med)
        return out[0], out[1], max(4.0, s_pred)
```

Mã nguồn: SignTracker.update() — dòng 617–648 trong file .py

```
    def update(self, frame_idx, detections):
        still = []
        for t in self.active:
            (self.finished if self._leaving_frame(t, frame_idx) else still).append(t)
        self.active = still
        pairs = []
        for ti, t in enumerate(self.active):
            pcx, pcy, ps = t.predict(frame_idx)
            gap = frame_idx - t.last_frame
            for di, d in enumerate(detections):
                if d["shape"] != t.shape:
                    continue
                cx, cy, s = _box_to_cs(d["box"])
                dist = math.hypot(cx - pcx, cy - pcy) / max(ps, 8.0)
                scale = max(s, ps) / max(1.0, min(s, ps))
                if dist <= TRK_GATE_DIST + 0.05 * gap and scale <= TRK_GATE_SCALE:
                    pairs.append((dist + 0.5 * (scale - 1.0), ti, di))
        pairs.sort()
        used_t, used_d = set(), set()
        for _, ti, di in pairs:
            if ti in used_t or di in used_d:
                continue
            used_t.add(ti); used_d.add(di)
            self.active[ti].add(frame_idx, detections[di])
        for di, d in enumerate(detections):
            if di not in used_d:
                self.active.append(Track(self.next_id, frame_idx, d))
                self.next_id += 1
        still = []
        for t in self.active:
            (still if frame_idx - t.last_frame <= TRK_MAX_GAP else self.finished).append(t)
        self.active = still
```

1.8 Bước 7 – Hậu xử lý track: phân loại, nối mảnh, khử trùng

Sau lượt 1, video 1 sinh ra khoảng 3752 track thô (đa số là nhiễu chỉ tồn tại vài frame). Các bước lọc:

1) **Lọc mảnh:** chỉ giữ track có ≥ 4 quan sát và mật độ quan sát ≥ 40% thời gian tồn tại (còn lại 692 mảnh).
2) **Nối mảnh:** hai mảnh cùng dạng hình được nối nếu mảnh sau bắt đầu ngay khi mảnh trước kết thúc (chồng lấn ≤ 10 frame) và vị trí khớp; hoặc cách nhau ≤ 30 frame và mảnh sau nằm đúng vị trí dự đoán của mảnh trước; hoặc cách nhau ≤ 150 frame nhưng gần như đứng yên (xe dừng đèn đỏ).

3) **Phân loại track** dựa trên **trung vị** của đặc trưng trên toàn track (bền vững hơn quyết định từng frame): ≥ 15 quan sát; tâm nằm ở phần giữa/phải (x ≥ 33% chiều rộng) và dải 20–50% chiều cao; đạt đường kính ≥ 18 px ít nhất một lần; **to dần theo thời gian** (hệ số góc của log(bán kính) theo frame > 0 và tương quan ≥ 0,4 – biển đứng yên bên đường luôn to dần khi xe tiến tới, còn nhiễu thì không). Sau đó mỗi dạng hình có tiêu chí riêng tính trên 30% quan sát lớn nhất: biển tròn cần độ tròn ≥ 0,90 cùng các điều kiện về độ lấp elip, màu lòng biển, vùng xung quanh; biển tam giác cần viền đỏ (≥ 20%) và ký hiệu tối trong lòng vàng (3–45%); biển chữ nhật cần ≥ 40 quan sát, nền xanh ≥ 40%, có chữ/ký hiệu trắng (6–60%) và kích thước ≤ 260 px (loại bảng quảng cáo lớn).

4) **Khử trùng:** các track được sắp theo độ dài; một track bị bỏ nếu ở ≥ 50% số frame chung, box của nó nằm ≥ 50% trong box của track đã giữ. Ở từng frame, box nằm trong box khác cũng bị bỏ – mỗi biển chỉ có **một** box.

Kết quả: video 1 còn 26 track, video 2 còn 5 track được xác nhận là biển báo.

Mã nguồn: SignTracker._is_traffic_sign() — dòng 675–723 trong file .py

```
    @classmethod
    def _is_traffic_sign(cls, t, fw, fh):
        """Phan loai 1 track da noi: vi tri + do ben + hinh dang luc gan + mau/noi dung."""
        dets = [d for _, d in t.obs]
        n = len(dets)
        if n < TRACK_MIN_OBS:
            return False
        med = lambda k, ds=dets: float(np.median([d[k] for d in ds]))

        # Đã cập nhật dòng này: Thêm điều kiện med("cy") / fh < TRACK_Y_MIN
        if med("cx") / fw < TRACK_X_MIN or med("cy") / fh < TRACK_Y_MIN or med("cy") / fh >
TRACK_Y_MAX:
            return False

        if 2 * max(d["r"] for d in dets) < TRACK_MIN_SIZE:
            return False
        if GROWTH_CHECK and not cls._grows_over_time(t.obs):
            return False
        near = sorted(dets, key=lambda d: d["r"])[int(n * 0.7):]
        if t.shape == "tri":
            # long vang + vien do + co ky hieu toi ben trong (tinh tren patch gan nhat)
            return (med("tri_red", near) >= TRI_RED_MIN
                    and TRI_DARK[0] <= _patch_median(t, _tri_symbol_frac) <= TRI_DARK[1])
        if t.shape == "rect":
            if n < RECT_MIN_OBS:
                return False
            # nen xanh, co chu / ky hieu trang, khong qua lon (bang quang cao)
            return (RECT_WHITE[0] <= _patch_median(t, _rect_white_frac) <= RECT_WHITE[1]
                    and med("rect_blue", near) >= RECT_BLUE_TRACK
                    and 2 * max(d["r"] for d in dets) <= RECT_MAX_SIZE
                    and med("mean_v", near) >= BLUE_MEANV_MIN)
        efill, ar = med("efill", near), med("ar", near)
        if med("circ", near) < TRACK_NEAR_CIRC:
            return False
        if med("border_red") >= med("border_blue"):
            return (RED_NEAR_EFILL[0] <= efill <= RED_NEAR_EFILL[1]
```

```
                    and RED_NEAR_AR[0] <= ar <= RED_NEAR_AR[1]
                    and med("inner_other") >= RED_INNER_MIN
                    and med("inner_wb") >= RED_INNER_WB_MIN
                    and med("mean_v") >= RED_MEANV_MIN
                    and med("outer_dark") <= RED_OUTER_DARK_MAX
                    and med("outer_blue") <= RED_OUTER_BLUE_MAX
                    and med("outer_red") <= RED_OUTER_RED_MAX)
        if not (BLUE_NEAR_EFILL[0] <= efill <= BLUE_NEAR_EFILL[1]
                and BLUE_NEAR_AR[0] <= ar <= BLUE_NEAR_AR[1]
                and med("mean_v") >= BLUE_MEANV_MIN):
            return False
        pure = med("border_blue", near) >= BLUE_PURE_BORDER and med("hue_med") >=
BLUE_PURE_HUE
        ringed = med("border_red", near) >= BLUE_RING_RED
        return pure or ringed
```

Mã nguồn: SignTracker._grows_over_time() — dòng 658–673 trong file .py

```
    @staticmethod
    def _grows_over_time(dets_obs):
        """
        [E] Bien tren xe dang tien toi phai to dan: log(r) tang tuyen tinh theo frame.
        Tra ve True neu he so goc > 0 va tuong quan >= GROWTH_MIN_CORR.
        Track qua ngan (< GROWTH_MIN_SPAN frame) thi bo qua kiem tra.
        """
        fr = np.array([f for f, _ in dets_obs], dtype=np.float64)
        if np.ptp(fr) < GROWTH_MIN_SPAN:
            return True
        ls = np.log(np.array([max(1.0, d["r"]) for _, d in dets_obs], dtype=np.float64))
        if np.std(ls) < 1e-9 or np.std(fr) < 1e-9:
            return False
        slope = np.polyfit(fr, ls, 1)[0]
        corr = float(np.corrcoef(fr, ls)[0, 1])
        return slope > 0 and corr >= GROWTH_MIN_CORR
```

1.9 Bước 8 – Nhận dạng nội dung biển báo

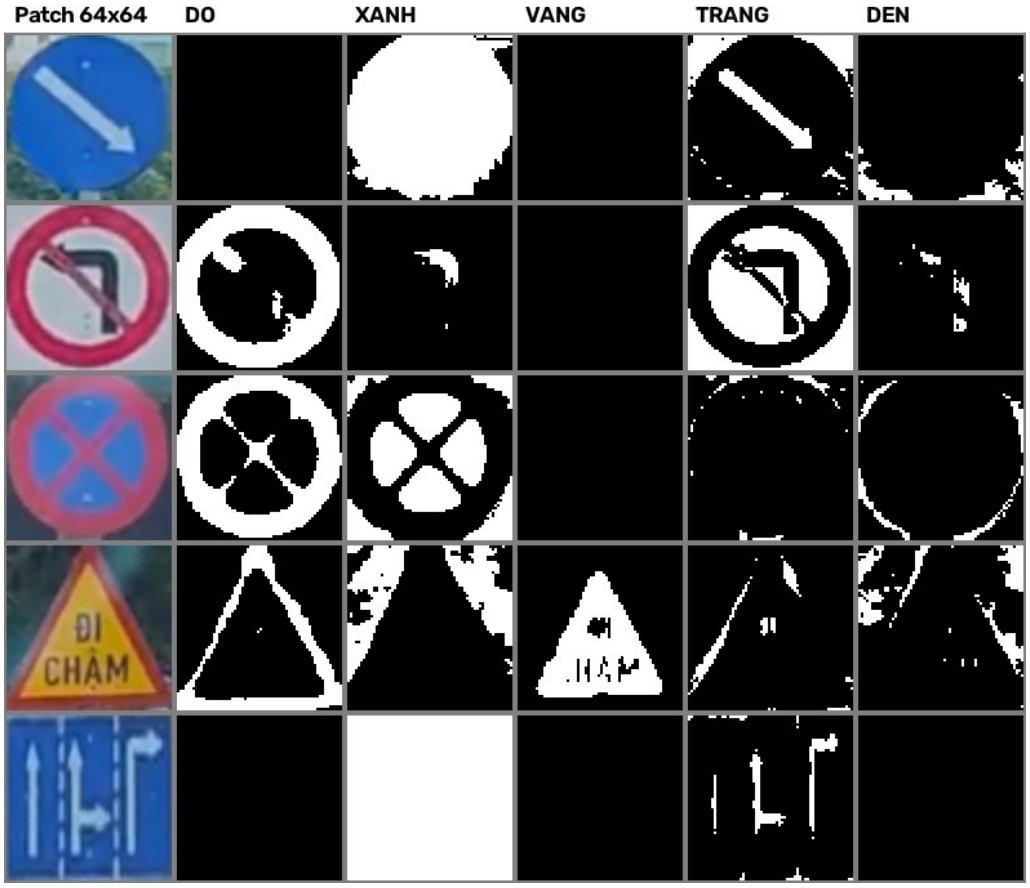
1.9.1 Tách mask màu trên ảnh chuẩn hoá

Mọi patch 64×64 được tách thành năm mask: ĐỎ, XANH, VÀNG (ngưỡng HSV tuyệt đối) và TRẮNG, ĐEN (ngưỡng **tương đối**). Video bị nén theo chroma subsampling nên nét trắng mảnh trên nền xanh vẫn có độ bão hoà khá cao: ngưỡng cố định S < 80 bỏ sót gần hết mũi tên. Vì vậy pixel được xem là trắng nếu S < 0,55 × S trung vị của vùng có màu và V ≥ 0,6 × V lớn nhất của patch; pixel đen nếu V < 0,6 × V trung vị của vùng có màu. Các vùng hình học (vành, lòng, dải chéo) được định nghĩa sẵn trên lưới toạ độ chuẩn hoá của patch.

Mã nguồn: patch_masks() — dòng 908–926 trong file .py

```
def patch_masks(patch):
    """
    Mask mau (bool) cua 1 patch BGR.
    TRANG / DEN lay nguong TUONG DOI so voi mau nen cua chinh bien (do/xanh/vang):
    video bi nen mau (chroma subsampling) nen net trang manh tren nen xanh van co
    do bao hoa kha cao -> nguong co dinh S < 80 se bo sot ky hieu.
```

```
    """
    hsv = cv2.cvtColor(patch, cv2.COLOR_BGR2HSV)
    h, s, v = hsv[:, :, 0], hsv[:, :, 1].astype(np.int32), hsv[:, :, 2].astype(np.int32)
    red = ((h <= 10) | (h >= 150)) & (s >= 70) & (v >= 50)
    blue = (h >= 95) & (h <= 135) & (s >= 70) & (v >= 40)
    yellow = (h >= YEL_H_MIN) & (h <= YEL_H_MAX) & (s >= 90) & (v >= 100)
    col = red | blue | yellow
    s_col = float(np.median(s[col])) if col.sum() > 20 else 150.0
    v_col = float(np.median(v[col])) if col.sum() > 20 else 150.0
    vmax = float(np.percentile(v, 97))
    white = (s < max(80.0, 0.55 * s_col)) & (v >= max(110.0, 0.60 * vmax))
    black = (v < max(60.0, 0.60 * v_col)) & ~col
    return dict(red=red, blue=blue, yellow=yellow, white=white, black=black)
```



Hình 1.8: Patch chuẩn hoá và 5 mask màu của các loại biển: hiệu lệnh, cấm rẽ, cấm dừng đỗ, cảnh báo, chỉ dẫn làn

1.9.2 Biển tròn: phân nhóm theo màu và so khớp mẫu

Cây quyết định cho biển tròn:

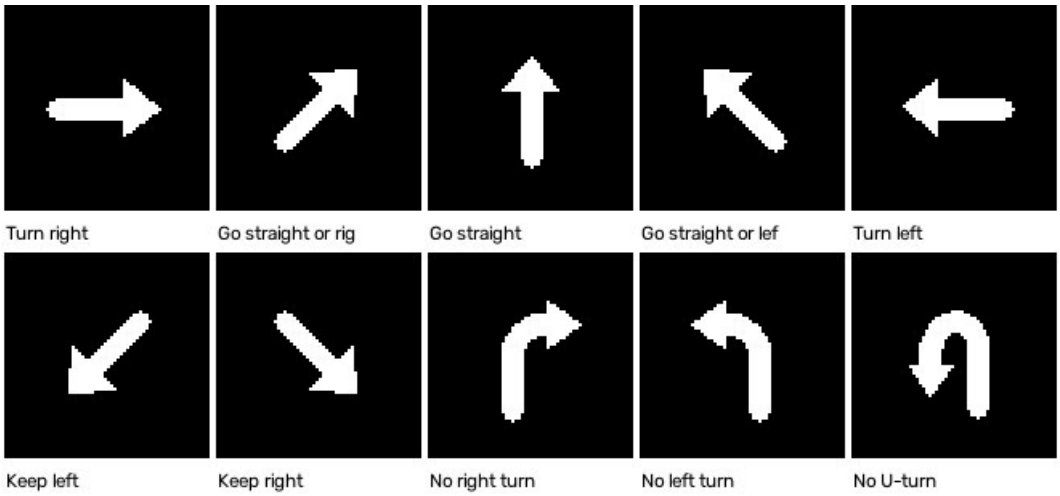
- **Vành đỏ, lòng đỏ, có vạch trắng nằm ngang** ở giữa (vùng |v| < 0,2 trắng > 35%, vùng trên/dưới vạch đỏ > 45%) → **No entry** (cấm đi ngược chiều).
- **Vành đỏ, lòng xanh** → biển cấm đỗ/dừng. Đĩa xanh được định vị lại trong patch (hợp các mảnh xanh lớn lân cận – vạch đỏ chia đĩa thành 2 hoặc 4 mảnh), rồi đo tỉ lệ đỏ trên hai dải chéo "\" và "/" (bỏ phần tâm nơi hai vạch giao nhau). Cả hai dải

đều đỏ → **No stopping & parking** (cấm dừng và đỗ, chữ X); chỉ một dải → **No parking** (cấm đỗ).

– **Vành đỏ, lòng trắng có ký hiệu tối** → biển cấm rẽ: so khớp mask ký hiệu với các mẫu *mũi tên rẽ phải, rẽ trái, quay đầu* (bỏ dải vạch chéo đỏ khỏi phép so khớp).

– **Nền xanh không viền đỏ** → biển hiệu lệnh: so khớp mask trắng với **7 mẫu mũi tên** theo các hướng 0°, 45°, 90°, 135°, 180°, 225°, 315° (rẽ phải, đi thẳng hoặc rẽ phải, đi thẳng, …, hướng phải đi vòng sang phải – *Keep right*).

Các mẫu **không cắt từ video** mà được vẽ trực tiếp bằng `cv2.line`, `cv2.ellipse`, `cv2.fillConvexPoly` trong chương trình (Hình 1.9), nên không vi phạm quy định hardcode. Phép so khớp dùng `cv2.matchTemplate` với độ đo `TM_CCOEFF_NORMED` trên mask đã làm mờ Gauss 7×7; mẫu được cắt bớt 4 px mỗi cạnh để cho phép lệch ±4 px. Kết quả được chấp nhận nếu điểm ≥ 0,30, hoặc ≥ 0,15 nhưng hơn mẫu thứ hai ≥ 0,10 (biển bị tối/ngược sáng).



Hình 1.9: Bộ mẫu ký hiệu được vẽ bằng code: 7 mũi tên biển hiệu lệnh và 3 ký hiệu biển cấm rẽ

Mã nguồn: _match() — dòng 986–1000 trong file .py

```
def _match(mask, templates, valid):
    """
    So khop mask nhi phan voi cac mau bang cv2.matchTemplate (TM_CCOEFF_NORMED).
    Lam mo 2 anh de chiu lech nho; cat vien 4 px cua mau de cho phep dich +-4 px.
    valid: vung pixel duoc tinh (vd: bo dai vach cheo do che ky hieu).
    Tra ve (ten tot nhat, diem, do chenh voi mau thu 2).
    """
    img = cv2.GaussianBlur((mask & valid).astype(np.float32), (7, 7), 0)
    scores = []
    for name, t in templates:
        tt = cv2.GaussianBlur(((t > 0) & valid).astype(np.float32), (7, 7), 0)[4:-4, 4:-4]
        scores.append((float(cv2.matchTemplate(img, tt, cv2.TM_CCOEFF_NORMED).max()),
name))
    scores.sort(reverse=True)
    margin = scores[0][0] - scores[1][0] if len(scores) > 1 else scores[0][0]
    return scores[0][1], scores[0][0], margin
```

Mã nguồn: recognize_circle() — dòng 1039–1096 trong file .py

```
def recognize_circle(patch):
    """Bien tron: bien cam (vien do) hoac bien hieu lenh (nen xanh)."""
    m = patch_masks(patch)
    ring = (_D >= 0.72) & (_D <= 0.98)
    inner = _D <= 0.62
    ring_red, ring_blue = _frac(m["red"], ring), _frac(m["blue"], ring)
    in_red, in_blue = _frac(m["red"], inner), _frac(m["blue"], inner)
    in_white = _frac(m["white"], inner)
    if ring_red >= ring_blue or in_red > 0.35:
        # (a) Nen do + vach trang nam ngang -> Cam di nguoc chieu
        bar = (np.abs(_V) < 0.20) & (np.abs(_U) < 0.55)
        off_bar = (np.abs(_V) > 0.38) & (_D < 0.75)
        if (in_red > 0.30 and _frac(m["white"], bar) > 0.35
                and _frac(m["red"], off_bar) > 0.45):
            return "No entry", 1.0
        # (b) Nen xanh + vien do: dem vach do theo 2 duong cheo
        if in_blue > 0.25:
            # dinh vi lai dia xanh trong patch (box co the bi lech / dinh vat the do ben
canh)
            # (vach do chia dia xanh thanh 2 hoac 4 manh -> lay HCN bao cua cac manh xanh)
            n, _, st, cen = cv2.connectedComponentsWithStats(m["blue"].astype(np.uint8), 8)
            if n > 1:
                k = 1 + int(np.argmax(st[1:, cv2.CC_STAT_AREA]))
                reach = 0.9 * max(st[k, 2], st[k, 3])        # cac manh xanh lan can cua
dia
                big = np.array([st[i] for i in range(1, n)
                                if st[i, 4] >= 0.25 * st[k, 4]
                                and np.hypot(*(cen[i] - cen[k])) <= reach])
            else:
                big = np.array([[0, 0, PATCH, PATCH, 0]])
            x, y = big[:, 0].min(), big[:, 1].min()
            w = (big[:, 0] + big[:, 2]).max() - x
            h = (big[:, 1] + big[:, 3]).max() - y
            u = (np.arange(PATCH)[None, :] - (x + w / 2.0)) / max(1.0, w / 2.0)
            v = (np.arange(PATCH)[:, None] - (y + h / 2.0)) / max(1.0, h / 2.0)
            dd = np.sqrt(u ** 2 + v ** 2)
            disk = (dd < 0.75) & (dd > 0.40)                # bo tam (vach day giao nhau o
tam)
            d1 = _frac(m["red"], (np.abs(u - v) < 0.20) & disk)
            d2 = _frac(m["red"], (np.abs(u + v) < 0.20) & disk)
            lo, hi = min(d1, d2), max(d1, d2)
            if lo >= 0.12 and lo >= 0.45 * hi:
                return "No stopping & parking", 1.0   # 2 vach cheo (X)
            return "No parking", 1.0                  # 1 vach cheo
        # (c) Nen trang + ky hieu den + vach cheo -> bien cam re (so khop mau)
        vch = cv2.cvtColor(patch, cv2.COLOR_BGR2HSV)[:, :, 2].astype(np.float64)
        v_bg = float(np.percentile(vch[inner], 90))      # do sang nen trang (co the bi
toi)
        dark = (vch < 0.60 * v_bg) & ~m["red"] & ~m["blue"]
        if _frac(dark, inner) > 0.04:                    # nen sang + ky hieu toi
            valid = (_D < 0.80) & ~_SLASH
            name, sc, mg = _match(dark, _TURN_T, valid)
            if sc >= TEMPLATE_MIN_SCORE or (sc >= 0.15 and mg >= 0.10):
                return name, max(sc, 0.35)
            return "Prohibitory sign", 0.3
        return "Prohibitory sign", 0.2
    # (d) Bien hieu lenh nen xanh: so khop mui ten trang voi 7 huong
    if in_white > 0.05:
```

```
        name, sc, mg = _match(m["white"], _MANDATORY_T, _D < 0.80)
        if sc >= TEMPLATE_MIN_SCORE:
            return name, sc
    return "Mandatory sign", 0.2
```

1.9.3 Biển tam giác và biển chữ nhật

Biển cảnh báo: ký hiệu là các pixel tối hơn hẳn nền vàng (V < 0,7 × V trung vị của nền vàng) trong lòng tam giác. Biển chữ (“ĐI CHẬM”) có ký hiệu xếp thành ≥ **2 dải dòng** với nhiều thành phần liên thông nhỏ (chữ cái) → **Slow down**; biển hình vẽ có **một dải** gồm 1–4 khối đen lớn (hình hai trẻ em) → **Children crossing**; còn lại → *Warning sign*.

Biển chỉ dẫn chữ nhật: phân tích các thành phần liên thông của ký hiệu trắng. ≥ 2 nét dọc cao (> 40% chiều cao) trên biển rộng → **Lane direction** (hướng đi theo làn); có mũi tên lớn → **Direction sign** (biển chỉ hướng); một khối trắng đặc ở nửa trên (hình camera) và ≥ 3 khối chữ ở nửa dưới → **Camera enforcement zone**; có ô trắng lớn sát bên trái (hình minh hoạ) và chỉ có chữ → **Accident-prone area** (đoạn đường hay xảy ra tai nạn); còn lại → *Information sign*.

Mã nguồn: recognize_triangle() — dòng 1099–1122 trong file .py

```
def recognize_triangle(patch):
    """Bien canh bao tam giac: phan biet ky hieu CHU (2 dong) va HINH NGUOI."""
    m = patch_masks(patch)
    if _frac(m["yellow"], _TRI_IN) < 0.25:
        return "Warning sign", 0.2
    sym = _tri_symbol(patch, m)
    if sym.sum() < 8:
        return "Warning sign", 0.2
    rows = sym.sum(axis=1) > 0
    # dem so "dai dong" co ky hieu (bo khe 1 px)
    runs, prev, gap = 0, False, 0
    for r in rows:
        if r and not prev and gap != 1:
            runs += 1
        gap = 0 if r else gap + 1
        prev = r
    n_cc, _, stats, _ = cv2.connectedComponentsWithStats(sym.astype(np.uint8), 8)
    blobs = [s for s in stats[1:] if s[cv2.CC_STAT_AREA] >= 4]
    if runs >= 2 and len(blobs) >= 4:
        return "Slow down", 1.0                      # chu "DI CHAM" xep 2 dong
    big = max(s[cv2.CC_STAT_AREA] for s in blobs) if blobs else 0
    if runs == 1 and 1 <= len(blobs) <= 4 and big >= 0.03 * PATCH * PATCH:
        return "Children crossing", 0.8              # hinh ve (2 tre em): 1 khoi den lon
    return "Warning sign", 0.3
```

Mã nguồn: recognize_rectangle() — dòng 1125–1152 trong file .py

```
def recognize_rectangle(patch, ar, left_white=0.0):
    """Bien chu nhat nen xanh: phan biet theo VI TRI + HINH DANG vung trang."""
    m = patch_masks(patch)
    white = m["white"]
    # Phan tich thanh phan lien thong cua ky hieu trang
```

```
    n, _, st, _ = cv2.connectedComponentsWithStats(white.astype(np.uint8), 8)
    comps = [s for s in st[1:] if s[cv2.CC_STAT_AREA] >= 6]
    H = W = float(PATCH)
    tall = [s for s in comps if s[cv2.CC_STAT_HEIGHT] > 0.40 * H
            and s[cv2.CC_STAT_WIDTH] < 0.35 * W]
    if ar >= 1.4 and len(tall) >= 2:
        return "Lane direction", 1.0                # cac mui ten / vach doc chay het
chieu cao
    arrow = [s for s in comps if s[cv2.CC_STAT_HEIGHT] > 0.50 * H
             and s[cv2.CC_STAT_WIDTH] < 0.50 * W]
    if tall or arrow:
        return "Direction sign", 0.6                # co mui ten lon + chu (bien chi
huong)
    # O trang lon sat BEN TRAI bien (hinh minh hoa) + chi co chu trang -> canh bao tai nan
    if left_white > 0.45:
        return "Accident-prone area", 1.0
    # hinh camera: 1 khoi trang DAC (khong phai dong chu dai, mong) o nua tren
    icon = [s for s in comps if s[cv2.CC_STAT_TOP] + s[cv2.CC_STAT_HEIGHT] < 0.50 * H
            and 0.25 * W <= s[cv2.CC_STAT_WIDTH] <= 0.75 * W
            and s[cv2.CC_STAT_HEIGHT] >= 0.15 * H
            and s[cv2.CC_STAT_AREA] >= 0.30 * s[cv2.CC_STAT_WIDTH] * s[cv2.CC_STAT_HEIGHT]]
    text = [s for s in comps if s[cv2.CC_STAT_TOP] > 0.40 * H]
    if icon and len(text) >= 3:
        return "Camera enforcement zone", 0.8       # hinh camera o tren + chu o duoi
    return "Information sign", 0.3
```

1.9.4 Bỏ phiếu trên track

Ảnh biển ở xa rất mờ nên chỉ dùng tối đa **6 quan sát gần nhất** (box lớn nhất, đúng tỉ lệ) của track. Mỗi quan sát cho một nhãn kèm điểm tin cậy; nhãn có tổng điểm lớn nhất là nội dung của **cả track**. Nhờ vậy nội dung hiển thị **không nhấp nháy** giữa các frame và biển được gắn đúng nhãn ngay từ khi còn ở xa.

Mã nguồn: recognize_track() — dòng 1155–1170 trong file .py

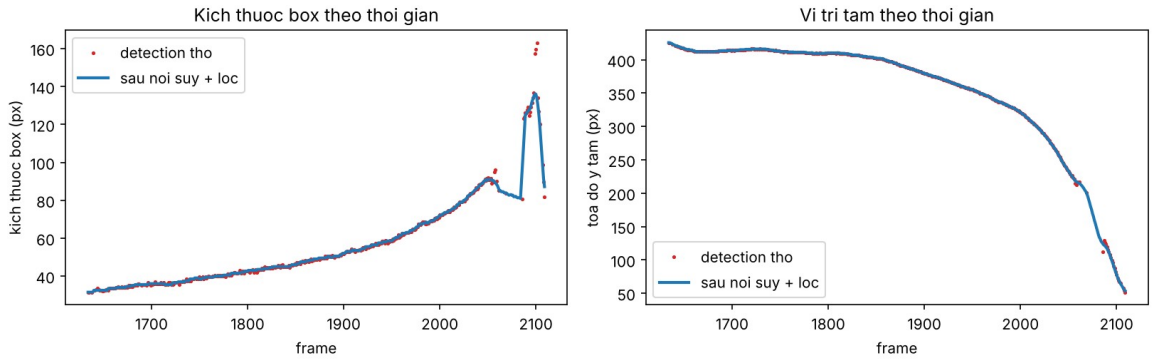
```
def recognize_track(t):
    """Bo phieu nhan dang tren cac patch lon nhat cua track (co trong so = diem)."""
    obs = sorted((d for _, d in t.obs if d.get("patch") is not None),
                 key=_patch_priority, reverse=True)[:PATCH_KEEP]
    votes = {}
    for d in obs:
        if t.shape == "circle":
            name, sc = recognize_circle(d["patch"])
        elif t.shape == "tri":
            name, sc = recognize_triangle(d["patch"])
        else:
            name, sc = recognize_rectangle(d["patch"], d["ar"], d.get("left_white", 0.0))
        votes[name] = votes.get(name, 0.0) + sc
    if not votes:
        return {"circle": "Traffic sign", "tri": "Warning sign"}.get(t.shape, "Information
sign")
    return max(votes.items(), key=lambda kv: kv[1])[0]
```

1.10 Bước 9 – Nội suy và làm mượt box

Với mỗi track được giữ, chương trình tạo box cho **mọi frame** từ frame đầu đến frame cuối: biểu diễn box bằng (tâm x, tâm y, kích thước √(w·h), tỉ lệ w/h); nội suy tuyến tính `np.interp` ở các frame mất detection; lọc **trung vị** cửa sổ 7 để bỏ các giá trị nhảy đột ngột; rồi **trung bình trượt** cửa sổ 5 để box chuyển động mượt. Hình 1.10 cho thấy box của biển cấm dừng đỗ trong video 1 trước và sau khi xử lý (440 detection được bù thành 476 frame liên tục).

Mã nguồn: SignTracker._dense_boxes() — dòng 815–841 trong file .py

```
    @staticmethod
    def _dense_boxes(t):
        """Box cho moi frame tu dau den cuoi track: noi suy + loc trung vi + trung binh
truot."""
        f0, f1 = t.obs[0][0], t.last_frame
        known_f = np.array([f for f, _ in t.obs], dtype=np.float64)
        known = np.array([_box_to_cs(d["box"]) + (d["box"][2] / float(max(1, d["box"]
[3])),)
                          for _, d in t.obs], dtype=np.float64)
        frames = np.arange(f0, f1 + 1)
        dense = np.stack([np.interp(frames, known_f, known[:, k]) for k in range(4)],
axis=1)
        if len(frames) >= MEDIAN_WIN:
            pad = MEDIAN_WIN // 2
            for c in range(4):
                col = np.pad(dense[:, c], pad, mode="edge")
                win = np.stack([col[i:i + len(frames)] for i in range(MEDIAN_WIN)], axis=1)
                dense[:, c] = np.median(win, axis=1)
        if len(frames) >= SMOOTH_WIN:
            k = np.ones(SMOOTH_WIN) / SMOOTH_WIN
            pad = SMOOTH_WIN // 2
            for c in range(4):
                col = np.pad(dense[:, c], pad, mode="edge")
                dense[:, c] = np.convolve(col, k, mode="valid")
        out = {}
        for f, (cx, cy, s, ar) in zip(frames, dense):
            w = s * math.sqrt(ar); h = s / math.sqrt(ar)
            out[int(f)] = (int(round(cx - w / 2)), int(round(cy - h / 2)),
                           int(round(w)), int(round(h)))
        return out
```



Hình 1.10: Kích thước và vị trí box của một track: điểm đỏ – detection thô, đường xanh – sau nội suy và làm mượt

1.11 Bước 10 – Ghi video kết quả và kiểm tra

Lượt 2 đọc lại video gốc, với mỗi frame vẽ box màu xanh lá (độ dày 2 px), ghi **nội dung biển phía trên box** (nếu sát mép trên thì ghi phía dưới) trên nền cùng màu để dễ đọc, và chèn chuỗi MSSV “52400062 - 52400200 - 52400063 - 52400248” vào góc trên trái. Cỡ chữ tự động theo chiều cao frame. Video được ghi bằng `cv2.VideoWriter` codec `mp4v`, **cùng kích thước, cùng FPS** với video gốc, frame BGR 3 kênh; tên file `52400062_52400200_52400063_52400248_video1.mp4` và `…_video2.mp4`. Cuối cùng hàm `verify_output` mở lại cả hai file để so sánh kích thước, FPS, số frame và số kênh màu.

Mã nguồn: draw_label() — dòng 1189–1216 trong file .py

```
def draw_label(frame, text, box, others=()):
    """
    Ghi noi dung bien bao ngay TREN box; neu sat mep tren hoac de len box cua bien
    khac (2 bien gan tren cung 1 cot) thi ghi DUOI box. Nen cung mau box, chu den.
    """
    fh, fw = frame.shape[:2]
    x, y, w, h = box
    scale = max(0.45, fh / 1080.0 * 0.62)
    thick = max(1, int(round(fh / 1080.0 * 1.6)))
    (tw, th), base = cv2.getTextSize(text, cv2.FONT_HERSHEY_SIMPLEX, scale, thick)
    pad = 3
    lw, lh = tw + 2 * pad, th + base + 2 * pad
    tx = min(max(0, x + w // 2 - lw // 2), max(0, fw - lw))

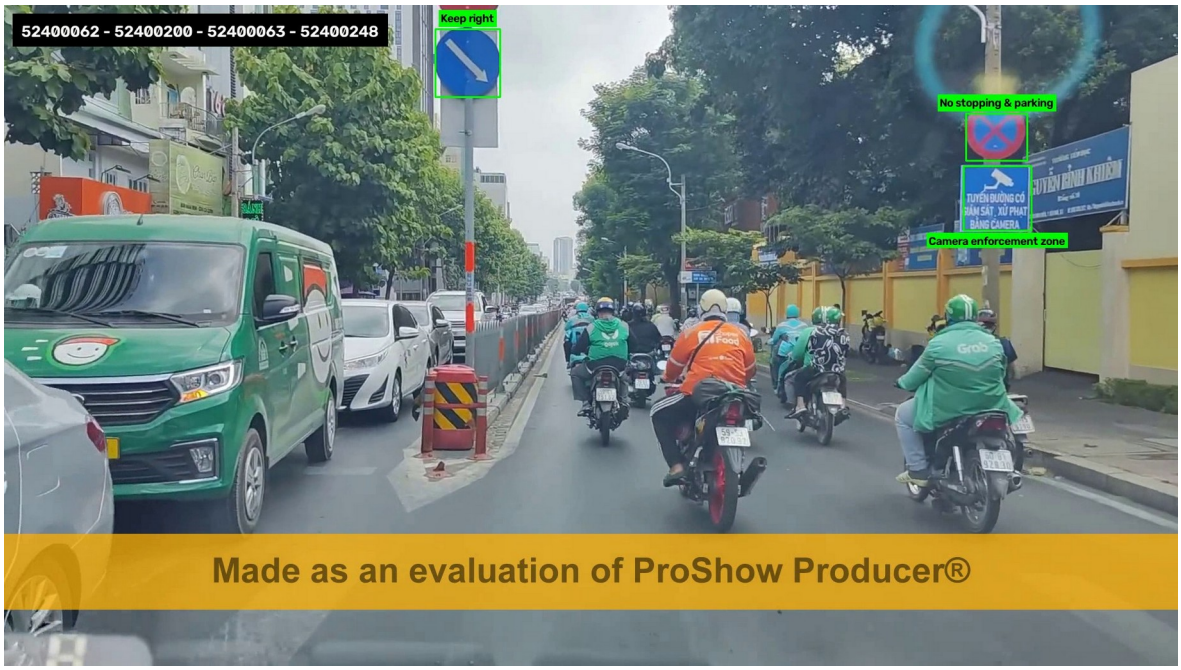
    def hits(ty):
        r = (tx, ty, lw, lh)
        return any(iou_min(r, o) > 0.15 for o in others)

    above, below = y - 4 - lh, y + h + 4
    if above >= 0 and not hits(above):
        ty = above
    elif below + lh <= fh and not hits(below):
        ty = below
    else:
        ty = max(0, above) if above >= 0 else min(below, fh - lh)
    cv2.rectangle(frame, (tx, ty), (tx + lw, ty + lh), BOX_COLOR, -1)
    cv2.putText(frame, text, (tx + pad, ty + pad + th), cv2.FONT_HERSHEY_SIMPLEX, scale,
                (0, 0, 0), thick, cv2.LINE_AA)
```

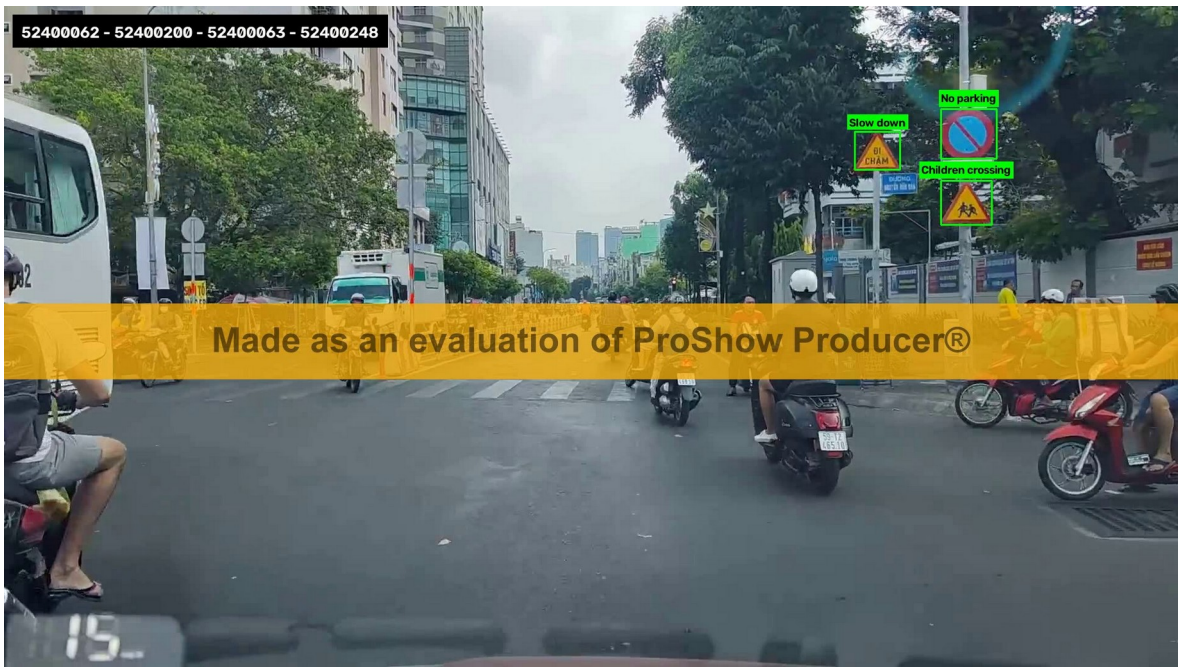
Mã nguồn: process_video() — dòng 1255–1319 trong file .py

```
    # ---------------- LUOT 2: ve + ghi video ----------------
    if DEBUG_OVERLAY:
        output_path = "debug_" + output_path
    cap = cv2.VideoCapture(input_path)
    writer = open_writer(output_path, fps, (fw, fh))
    written = 0
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        if DEBUG_OVERLAY:
            for (x, y, w, h) in tracker.rejected.get(written, []):
```

```
                cv2.rectangle(frame, (x, y), (x + w, y + h), (0, 255, 255), 1)
        items = boxes_per_frame.get(written, [])
        for (x, y, w, h), _ in items:
            cv2.rectangle(frame, (x, y), (x + w, y + h), BOX_COLOR, 2)
        for i, (b, label) in enumerate(items):
            draw_label(frame, label, b, [o for j, (o, _) in enumerate(items) if j != i])
        draw_student_ids(frame, ids_text)
        writer.write(frame)
        written += 1
        if SHOW_PREVIEW:
            cv2.imshow("preview", frame)
            cv2.waitKey(1)
    cap.release()
    writer.release()
    if SHOW_PREVIEW:
        cv2.destroyAllWindows()
    if written != n_frames:
        print("  CANH BAO: luot 2 ghi %d frame, luot 1 doc %d frame" % (written, n_frames))
    print("  Da ghi %s (%d frame, %.0fs)" % (output_path, written, now_seconds() - t0))
    verify_output(input_path, output_path)
    return tracker
```



Hình 1.11: Một frame kết quả của video 1: box, nội dung biển và MSSV



Hình 1.12: Một frame kết quả của video 2

CHƯƠNG 2 – KẾT QUẢ THỰC HIỆN

2.1 Tổng hợp kết quả

Chương trình được chạy với cùng mã nguồn và bộ tham số cho cả hai video. Bảng 2.1 liệt kê các biển báo mà hệ thống phát hiện và nội dung nhận dạng được (thời điểm tính theo giây trong video kết quả).

Bảng 2.1: Các biển báo được phát hiện và nhận dạng

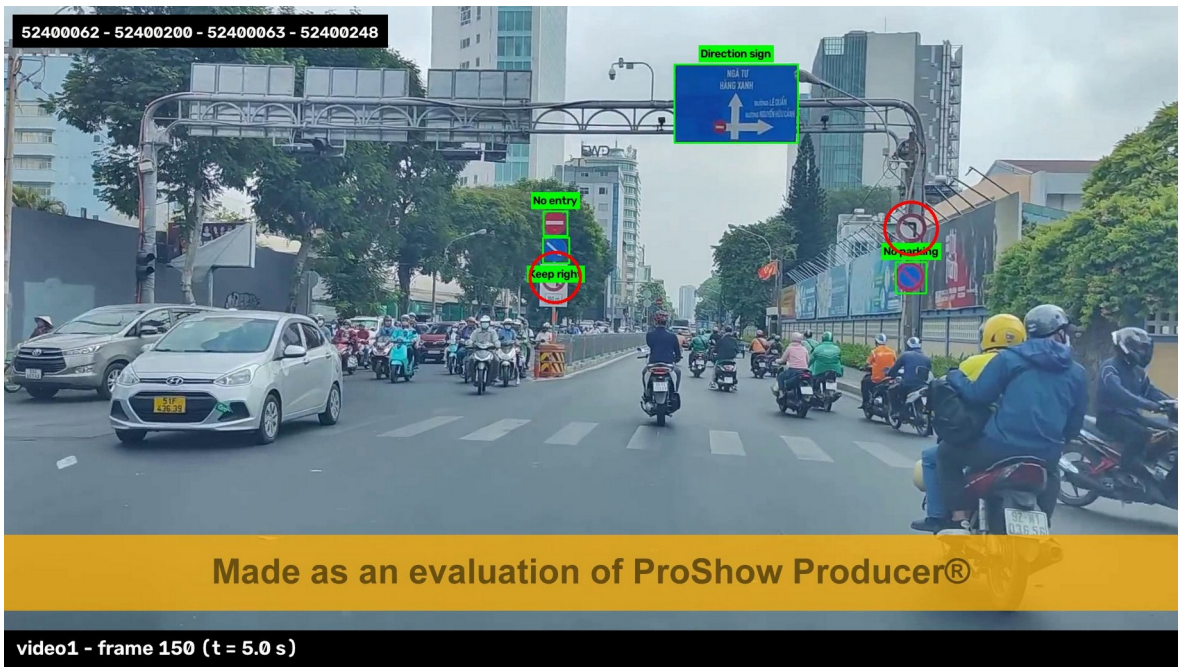
STT	Video	Thời gian (s)	Dạng	Nội dung hiển thị
1	Video 1	0.0 – 8.9	Tròn	Keep right (Hướng phải đi vòng sang phải)
2	Video 1	0.0 – 8.5	Tròn	No entry (Cấm đi ngược chiều)
3	Video 1	0.0 – 8.5	Tròn	No parking (Cấm đỗ xe)
4	Video 1	0.0 – 6.8	Chữ nhật	Direction sign (Biển chỉ hướng đi)
5	Video 1	5.6 – 9.3	Tròn	No left turn (Cấm rẽ trái)
6	Video 1	6.1 – 7.1	Tròn	No left turn (Cấm rẽ trái)
7	Video 1	12.6 – 19.3	Tròn	No entry (Cấm đi ngược chiều)
8	Video 1	12.7 – 17.7	Tròn	Keep right (Hướng phải đi vòng sang phải)
9	Video 1	20.6 – 28.4	Tròn	Keep right (Hướng phải đi vòng sang phải)
10	Video 1	20.9 – 28.0	Tròn	No entry (Cấm đi ngược chiều)
11	Video 1	24.0 – 27.5	Chữ nhật	Lane direction (Hướng đi trên mỗi làn)
12	Video 1	24.2 – 35.7	Tròn	Keep right (Hướng phải đi vòng sang phải)
13	Video 1	25.0 – 35.3	Tròn	No entry (Cấm đi ngược chiều)
14	Video 1	31.6 – 43.9	Tròn	Keep right (Hướng phải đi vòng sang phải)
15	Video 1	33.1 – 43.4	Tròn	No entry (Cấm đi ngược chiều)
16	Video 1	40.3 – 69.4	Tròn	Keep right (Hướng phải đi vòng sang phải)
17	Video 1	40.8 – 42.4	Tròn	No left turn (Cấm rẽ trái)
18	Video 1	41.3 – 68.4	Tròn	No entry (Cấm đi ngược chiều)
19	Video 1	45.3 – 48.2	Tròn	No left turn (Cấm rẽ trái)
20	Video 1	54.5 – 70.4	Tròn	No stopping & parking (Cấm dừng xe và đỗ xe)
21	Video 1	65.2 – 70.3	Chữ nhật	Camera enforcement zone (Tuyến đường có camera giám sát)
22	Video 1	73.2 – 77.3	Chữ nhật	Accident-prone area (Đoạn đường hay xảy ra tai nạn)
23	Video 1	82.0 – 83.8	Tam giác	Slow down (Đi chậm)
24	Video 1	83.9 – 92.2	Tròn	No entry (Cấm đi ngược chiều)

STT	Video	Thời gian (s)	Dạng	Nội dung hiển thị
25	Video 1	84.1 – 92.5	Tròn	Keep right (Hướng phải đi vòng sang phải)
26	Video 1	86.6 – 91.6	Tròn	No parking (Cấm đỗ xe)
27	Video 2	2.6 – 5.6	Tròn	No parking (Cấm đỗ xe)
28	Video 2	11.8 – 23.1	Tròn	No parking (Cấm đỗ xe)
29	Video 2	16.7 – 23.1	Tam giác	Children crossing (Trẻ em)
30	Video 2	20.5 – 23.7	Tam giác	Slow down (Đi chậm)
31	Video 2	31.8 – 63.4	Tròn	No parking (Cấm đỗ xe)

Tổng cộng hệ thống xuất ra 31 biển báo (26 ở video 1, 5 ở video 2), mỗi biển có đúng một box xuyên suốt từ lúc xuất hiện đến khi ra khỏi khung hình. Nhóm đã kiểm tra ảnh của từng track trong số 31 track này: tất cả đều là biển báo thật và nội dung hiển thị khớp với biển (không có track nào nằm trên vật thể không phải biển báo). Kết quả output đúng yêu cầu định dạng: cùng kích thước 1920×1080, cùng 29,97 fps, cùng số frame (3087 và 2278) và cùng hệ màu BGR 3 kênh với video gốc (hàm verify_output báo OK cho cả 4 tiêu chí). Những trường hợp chưa chính xác còn lại được trình bày trong các mục 2.2 và 2.3.

2.2 Các frame nhận dạng chưa chính xác – Video 1

Frame 150 (t = 5.0 s): hai biển “Cấm rẽ trái” (khoanh đỏ) – một biển ở cột giữa đường, một biển ở cột bên phải – **chưa có box**. Ở thời điểm này biển còn nhỏ, nền trắng bị ngược sáng nên viền đỏ mảnh lúc được lúc không trên mask đỏ; các mảnh track ở giai đoạn xa không đạt điều kiện lọc (độ tròn, mật độ quan sát) nên bị loại. Box chỉ xuất hiện từ frame 168 (biển ở cột giữa) và frame 183 (biển bên phải) khi biển đủ lớn. Các biển còn lại trong frame (cấm đi ngược chiều, hướng phải đi vòng sang phải, cấm đỗ xe, biển chỉ hướng trên giá long môn) đều được nhận dạng đúng.



Hình 2.1: Video 1, frame 150 (t = 5.0 s): hai biển cấm rẽ trái chưa được phát hiện (khoanh đỏ)

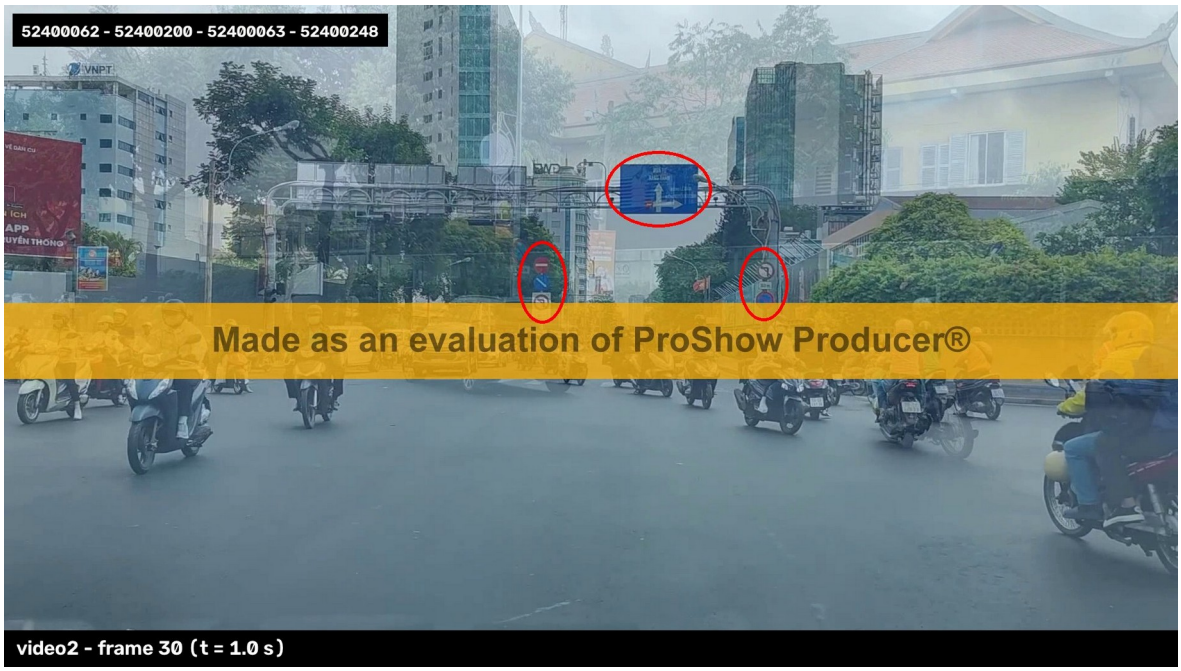
Frame 2280 (t = 76.1 s): biển phụ chữ nhật nền đỏ "CHÚ Ý QUAN SÁT" **không được phát hiện**. Hệ thống chỉ tìm biển chữ nhật nền xanh; biển chữ nhật nền đỏ bị bỏ qua có chủ đích vì dễ nhầm với các bảng hiệu quảng cáo nền đỏ xuất hiện dày đặc trong video (ngân hàng, cửa hàng). Biển "Đoạn đường hay xảy ra tai nạn" ngay bên dưới được phát hiện và nhận dạng đúng.



Hình 2.2: Video 1, frame 2280 (t = 76.1 s): biển "CHÚ Ý QUAN SÁT" nền đỏ không được phát hiện

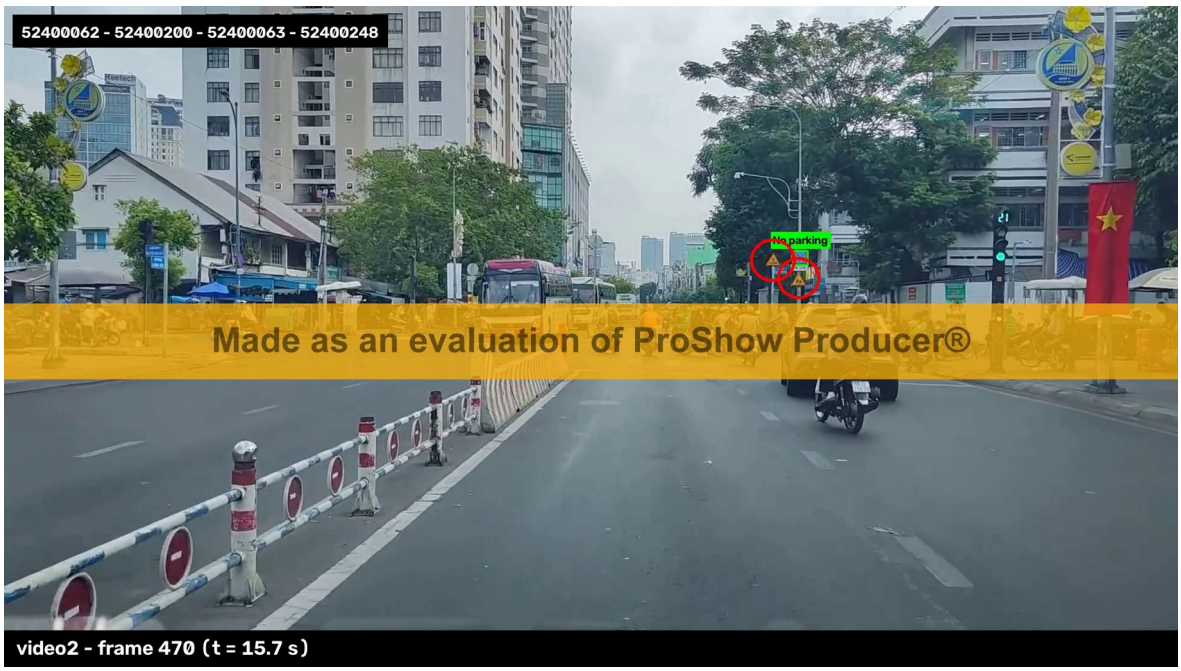
2.3 Các frame nhận dạng chưa chính xác – Video 2

Frame 30 (t = 1.0 s): đầu video 2 là đoạn chuyển cảnh (hai cảnh chồng mờ lên nhau). Các biển trong đoạn này – biển chỉ hướng trên giá long môn, cụm biển ở cột giữa và cụm biển bên phải – **không có box**: ảnh bị hoà trộn làm màu nhạt đi (độ bão hoà giảm) và cảnh chỉ kéo dài khoảng 1,3 s, không đủ số quan sát tối thiểu để track được xác nhận.



Hình 2.3: Video 2, frame 30 (t = 1.0 s): các biển trong đoạn chuyển cảnh không được phát hiện

Frame 470 (t = 15.7 s): biển cấm đỗ xe ở xa đã có box và nhãn đúng, nhưng hai biển cảnh báo tam giác "Đi chậm" và "Trẻ em" bên dưới (khoanh đỏ) **chưa có box**: lòng vàng của chúng lúc này chỉ rộng ≈ 15–20 px nên `approxPolyDP` chưa cho được đúng 3 đỉnh. Box của hai biển này chỉ xuất hiện từ frame 501 và 613 khi biển đủ lớn.



Hình 2.4: Video 2, frame 470 (t = 15.7 s): hai biển cảnh báo ở xa chưa được phát hiện

2.4 Đường dẫn video kết quả

Hai video kết quả được tải lên Google Drive và chia sẻ công khai (bất kỳ ai có đường dẫn đều xem được):

- **Video 1 (52400062_52400200_52400063_52400248_video1.mp4):** [Dán link Google Drive video 1]
- **Video 2 (52400062_52400200_52400063_52400248_video2.mp4):** [Dán link Google Drive video 2]

2.5 Nhận xét và hướng phát triển

- **Ưu điểm:** cùng một mã nguồn và bộ tham số chạy đúng trên cả hai video; box ổn định và duy nhất nhờ theo dõi + lọc theo cả track + nội suy, làm mượt; nội dung được quyết định một lần cho mỗi track nên không nhấp nháy; nhờ ảnh phóng to ×2, ×3 mà box xuất hiện khi biển còn ở khá xa.
- **Hạn chế về phát hiện:** chỉ xét ba màu đỏ/xanh/vàng với hình tròn, tam giác và chữ nhật nền xanh; biển chữ nhật nền đỏ (“CHÚ Ý QUAN SÁT”) chưa được phát hiện vì dễ nhầm với biển hiệu quảng cáo màu đỏ (ngân hàng, cửa hàng). Biển chỉ dẫn thấy trong thời gian ngắn (< 40 frame) bị bỏ để tránh box thừa trên bảng hiệu.
- **Hạn chế về độ ổn định:** biển rất xa (bán kính < 4 px) hoặc biển trắng viền đỏ bị ngược sáng có thể chỉ được nhận ra muộn; khi biển ra sát mép khung hình, box có thể bị lệch vài frame.
- **Hạn chế về nhận dạng:** nội dung chữ (ĐI CHẬM, camera, tai nạn) được suy ra từ bố cục ký hiệu chứ không đọc chữ (OCR), nên chỉ phân biệt được các loại biển

có bố cục khác nhau; các biển lạ ngoài bộ mẫu sẽ nhận nhãn chung (Prohibitory / Mandatory / Warning / Information sign).

- **Hướng phát triển:** bổ sung phát hiện biển chữ nhật nền đỏ/trắng kèm kiểm tra cột đỡ; dùng đặc trưng hình dạng bất biến (Hu moments) hoặc so khớp đặc trưng (ORB + BFMatcher) cho bộ ký hiệu lớn hơn; ước lượng chuyển động camera (optical flow) để dự đoán vị trí track tốt hơn khi xe rẽ.

TÀI LIỆU THAM KHẢO

[1] R. C. Gonzalez, R. E. Woods, *Digital Image Processing*, 4th ed., Pearson, 2018.

[2] OpenCV Documentation, “Changing Colorspaces”, “Morphological Transformations”, “Contours: Getting Started / Contour Features”, “Template Matching”, “Histograms – CLAHE”, https://docs.opencv.org/4.x/.

[3] C. Tomasi, R. Manduchi, “Bilateral filtering for gray and color images,” *Proc. ICCV*, 1998, pp. 839–846.

[4] K. Zuiderveld, “Contrast Limited Adaptive Histogram Equalization,” in *Graphics Gems IV*, Academic Press, 1994, pp. 474–485.

[5] Bộ Giao thông Vận tải, *QCVN 41:2019/BGTVT – Quy chuẩn kỹ thuật quốc gia về báo hiệu đường bộ*, 2019.

[6] Tài liệu hướng dẫn xử lý video (Guideline for Video processing in Midterm and Final exams), môn Nhập môn Xử lý ảnh số 505060, Khoa CNTT, ĐH Tôn Đức Thắng.